

# Package ‘mathmodels’

June 30, 2026

**Type** Package

**Title** Comprehensive Mathematical Modeling Algorithms in R

**Version** 0.0.9

**Author** Jingxin Zhang

**Maintainer** Jingxin zhang <zhjx\_19@163.com>

**Description** A versatile R package for mathematical modeling, developed as a companion to Mathematical Modeling: Algorithms and Programming Implementation (China Machine Press). The package implements algorithms across differential and difference equations, statistical analysis, optimization, evaluation, and prediction. Currently, it focuses on evaluation algorithms, including indicator data preprocessing (e.g., standardization, rescaling), subjective and objective weighting methods (e.g., AHP, entropy weighting, CRITIC, PCA weighting) and weight combination, TOPSIS, fuzzy comprehensive evaluation, grey relational analysis, rank sum ratio, DEA (CCR, BCC, SBM, Malmquist, powered by lpSolveAPI), regional economics (LQ, HHI and EG index), inequality measures (e.g., Gini and Theil indices), grey prediction models (e.g., GM(1,1), GM(1,N), Verhulst), Markov chain prediction, and Grey-Markov combined prediction. It also provides ordinary differential equation (ODE) solvers for compartmental epidemic models (SIR, SEIR, SEIRS, SIDP), along with epidemic visualization and summary metrics. Designed for researchers and practitioners in mathematical modeling.

**License** AGPL (>= 3)

**URL** <https://github.com/zhjx19/mathmodels>, <https://zhjx19.github.io/mathmodels/>

**BugReports** <https://github.com/zhjx19/mathmodels/issues>

**Encoding** UTF-8

**LazyData** true

**Roxygen** list(markdown = TRUE)

**Imports** lpSolveAPI,  
dplyr (>= 1.0.0),  
forecast,  
ggplot2 (>= 3.5.0),  
MASS,  
purrr,  
readxl,  
rlang,  
stats,  
stringr,  
tibble,  
tidyr (>= 1.1.0),

tseries,  
deSolve,  
patchwork,  
rugarch

**Depends** R (>= 4.1)

**Suggests** knitr,  
quarto,  
rmarkdown,  
scales,  
testthat (>= 3.0.0)

**VignetteBuilder** knitr

**Config/testthat/edition** 3

**Config/roxygen2/version** 8.0.0

## Contents

|                             |    |
|-----------------------------|----|
| AHP . . . . .               | 3  |
| combine_preds . . . . .     | 4  |
| combine_weights . . . . .   | 4  |
| compute_mf . . . . .        | 5  |
| critic_weight . . . . .     | 6  |
| cv_weight . . . . .         | 7  |
| DEA . . . . .               | 8  |
| defuzzify . . . . .         | 10 |
| entropy_weight . . . . .    | 11 |
| epi_metrics . . . . .       | 12 |
| fuzzy_eval . . . . .        | 13 |
| GM11_markov . . . . .       | 14 |
| grey_analysis . . . . .     | 15 |
| grey_models . . . . .       | 16 |
| inequality . . . . .        | 17 |
| linear_sum . . . . .        | 19 |
| markov_chain . . . . .      | 20 |
| membership . . . . .        | 21 |
| model_logistic . . . . .    | 24 |
| model_lv . . . . .          | 25 |
| model_malthus . . . . .     | 26 |
| model_seir . . . . .        | 27 |
| model_si . . . . .          | 28 |
| model_sir . . . . .         | 29 |
| model_sis . . . . .         | 30 |
| ode_solver . . . . .        | 31 |
| pca_weight . . . . .        | 32 |
| plot_compartments . . . . . | 33 |
| plot_incidence . . . . .    | 33 |
| plot_phase_si . . . . .     | 34 |
| plot_Rt_estimate . . . . .  | 35 |
| plot_ts . . . . .           | 35 |
| plot_ts_acf . . . . .       | 36 |
| plot_ts_forecast . . . . .  | 37 |

|                                |    |
|--------------------------------|----|
| plot_ts_garch . . . . .        | 38 |
| plot_ts_pacf . . . . .         | 38 |
| plot_ts_residuals . . . . .    | 39 |
| plot_ts_sarima_garch . . . . . | 39 |
| plot_ts_stl . . . . .          | 40 |
| preprocess . . . . .           | 40 |
| rank_sum_ratio . . . . .       | 43 |
| read_nbs . . . . .             | 44 |
| regional_economics . . . . .   | 44 |
| system_evaluation . . . . .    | 46 |
| topsis . . . . .               | 47 |
| ts_back_transform . . . . .    | 49 |
| ts_ets . . . . .               | 49 |
| ts_forecast . . . . .          | 50 |
| ts_garch . . . . .             | 51 |
| ts_sarima . . . . .            | 52 |
| ts_sarima_garch . . . . .      | 53 |
| ts_stl . . . . .               | 54 |
| ts_test . . . . .              | 54 |
| ts_transform . . . . .         | 55 |
| water_quality . . . . .        | 56 |

|              |           |
|--------------|-----------|
| <b>Index</b> | <b>58</b> |
|--------------|-----------|

AHP

*AHP: Analytic Hierarchy Process***Description**

AHP is a multi-criteria decision analysis method developed by Saaty, which can also be used to determine indicator weights.

**Usage**

```
AHP(A)
```

**Arguments**

A a numeric matrix, i.e. pairwise comparison matrix

**Value**

a list object that contains: w (Weight vector), CR (Consistency ratio), Lmax (Maximum eigenvalue), CI (Consistency index)

**Examples**

```
A = matrix(c(1, 1/2, 4, 3, 3,
              2, 1, 7, 5, 5,
              1/4, 1/7, 1, 1/2, 1/3,
              1/3, 1/5, 2, 1, 1,
              1/3, 1/5, 3, 1, 1), byrow = TRUE, nrow = 5)
AHP(A)
```

---

 combine\_preds

*Combine Multiple Prediction Results*


---

### Description

Combines multiple prediction results (e.g., from grey prediction, time series, or machine learning models) into a single prediction using a similarity-based weighting approach, improving prediction accuracy.

### Usage

```
combine_preds(x)
```

### Arguments

**x** Numeric vector, prediction results to be combined (length  $\geq 2$ ).

### Details

The function combines prediction results by constructing a similarity matrix based on cosine transformation of pairwise differences. Weights are derived from the principal eigenvector of the similarity matrix, ensuring predictions closer to each other have higher influence. For two predictions, equal weights (0.5, 0.5) are used. If all predictions are identical, equal weights are assigned. Compatible with the `mathmodels` package for enhancing prediction models, including grey prediction, time series, or ensemble machine learning.

### Value

A list with two elements:

- **a**: Numeric, the combined prediction value.
- **w**: Numeric vector, weights for each prediction in **x**, summing to 1.

### Examples

```
# Example: Combine three prediction results
preds = c(100, 102, 98) # E.g., from grey prediction, ARIMA, or ML models
combine_preds(preds)
```

---

 combine\_weights

*Combine Subjective and Objective Weights*


---

### Description

Combines subjective and objective weights using linear, multiplicative, or game theory-based methods (geometric mean or linear system).

### Usage

```
combine_weights(w_subj, w_obj, type = "linear", alpha = 0.5)
```

**Arguments**

|        |                                                                                                               |
|--------|---------------------------------------------------------------------------------------------------------------|
| w_subj | Numeric vector of subjective weights.                                                                         |
| w_obj  | Numeric vector of objective weights.                                                                          |
| type   | Character string specifying the combination method: "linear", "multiplicative", "game", or "game_linear".     |
| alpha  | Numeric value between 0 and 1, used only for the linear method to weight subjective weights. Defaults to 0.5. |

**Details**

The function supports four methods:

- Linear: Combines weights as  $\alpha * w_{\text{subj}} + (1 - \alpha) * w_{\text{obj}}$ .
- Multiplicative: Combines weights as  $w_{\text{subj}} * w_{\text{obj}}$ , requiring positive weights.
- Game: Uses the geometric mean ( $\sqrt{w_{\text{subj}} * w_{\text{obj}}}$ ) to balance weights.
- Game\_linear: Uses a game-theoretic approach by solving a linear system based on the cross-product of weights.

**Value**

A numeric vector of combined weights, normalized to sum to 1.

**Examples**

```
w_subj = c(0.4, 0.3, 0.2, 0.1)
w_obj = c(0.25, 0.2, 0.3, 0.25)
combine_weights(w_subj, w_obj, type = "linear", alpha = 0.6)
combine_weights(w_subj, w_obj, type = "multiplicative")
combine_weights(w_subj, w_obj, type = "game")
combine_weights(w_subj, w_obj, type = "game_linear")
```

---

|            |                                                                                       |
|------------|---------------------------------------------------------------------------------------|
| compute_mf | <i>Compute fuzzy membership vector and return corresponding membership functions.</i> |
|------------|---------------------------------------------------------------------------------------|

---

**Description**

compute\_mf transforms a single indicator value into a fuzzy membership vector, where each element represents the degree of membership to a specific evaluation level. compute\_mf\_funs returns the list of membership functions for visualization purposes.

**Usage**

```
compute_mf_funs(thresholds)

compute_mf(x, thresholds)
```

## Arguments

|            |                                                                                                                 |
|------------|-----------------------------------------------------------------------------------------------------------------|
| thresholds | A numeric vector containing at least two threshold values that define the boundaries between evaluation levels. |
| x          | A numeric scalar representing the value of an indicator.                                                        |

## Value

A list with two elements:

**mv** A numeric vector, membership degrees to each level.

**mfs** A list of functions, one per level, for plotting membership functions.

## Examples

```
# Example: SO2 concentration = 0.07, thresholds = c(0.05, 0.15, 0.25, 0.5)
th = c(0.05, 0.15, 0.25, 0.5)
compute_mf(0.07, th)

## Not run:
mfs = compute_mf_funs(th)
plots = lapply(mfs, \(x) plot_mf(x, xlim = c(0, 0.6)))
gridExtra::grid.arrange(grobs = plots, nrow = 2)

## End(Not run)
```

---

critic\_weight

---

*CRITIC Weight Method*


---

## Description

Computes objective weights of indicators and scores of samples using the CRITIC method. The method considers both the contrast intensity (e.g., standard deviation or entropy) and conflict among indicators (based on correlation) to determine indicator importance. This version supports different methods for contrast intensity and correlation types.

## Usage

```
critic_weight(
  X,
  index = NULL,
  method = "std",
  cor_method = "pearson",
  epsilon = 0.002
)
```

Arguments

|            |                                                                                                                                                                                                                                                                                                                                                                         |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| X          | A numeric data frame or matrix where rows represent samples (observations) and columns represent indicators (variables).                                                                                                                                                                                                                                                |
| index      | A character vector indicating the direction of each indicator: Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better), and NA for already normalized indicators (no rescaling will be applied).<br><br>If <code>`index = NULL`</code> (default), all indicators are treated as <code>`NA`</code> , meaning no normalization. |
| method     | Character scalar; specifies the method used to compute contrast intensity. Options: "std" (standard deviation, default), or "entropy" (based on information redundancy).                                                                                                                                                                                                |
| cor_method | Character scalar; specifies the method for computing correlations. Options: "pearson" (default), "spearman", or "kendall".                                                                                                                                                                                                                                              |
| epsilon    | A small constant used to replace exact 0s and 1s in the data to prevent log(0) errors. Default is 0.002. Only used when <code>method = "entropy"</code> .                                                                                                                                                                                                               |

Value

|                    |                                                                |
|--------------------|----------------------------------------------------------------|
| A list containing: |                                                                |
| w                  | Numeric vector of weights for each indicator.                  |
| s                  | Numeric vector of scores for each sample (row), scaled by 100. |

Examples

```
# Example: Using CRITIC method on a simple dataset
X = data.frame(
  x1 = c(3, 5, 2, 7),
  x2 = c(10, 20, 15, 25)
)
index = c("+", "-")
critic_weight(X, index)
critic_weight(X, index, method = "entropy")
```

---

|           |                                           |
|-----------|-------------------------------------------|
| cv_weight | <i>Coefficient of Variation Weighting</i> |
|-----------|-------------------------------------------|

---

Description

Computes weights for indicators using the Coefficient of Variation (CV) method. Weights are derived by normalizing the CV (standard deviation divided by mean) for each indicator.

Usage

```
cv_weight(X)
```

Arguments

|   |                                                            |
|---|------------------------------------------------------------|
| X | Numeric matrix or data frame with positive indicator data. |
|---|------------------------------------------------------------|

### Details

The `cv_weight` function calculates weights using the CV method. For each column in `data`, the CV is computed as the standard deviation divided by the mean. Weights are obtained by normalizing the CVs to sum to 1. This lightweight implementation uses base R and assumes all columns are numeric indicators.

### Value

Numeric vector of weights for the indicators, summing to 1.

### Examples

```
X = data.frame(x1 = c(10, 20, 15), x2 = c(5, 10, 8))
cv_weight(X)
```

---

DEA

---

*DEA efficiency analysis*


---

### Description

DEA efficiency analysis

### Usage

```
basic_DEA(
  data,
  inputs,
  outputs,
  ud_outputs = NULL,
  orientation = "io",
  rts = "vrs"
)

super_DEA(data, inputs, outputs, orientation = "io", rts = "vrs")

basic_SBM(
  data,
  inputs,
  outputs,
  ud_outputs = NULL,
  orientation = "io",
  rts = "vrs"
)

super_SBM(data, inputs, outputs, orientation = "io", rts = "vrs")

malmquist(
  data,
  period,
  inputs,
```



```

    outputs,
    orientation = "oo",
    rts = "vrs",
    type1 = "glob",
    type2 = "rd"
  )

```

### Arguments

|             |                                                                                                                                                                                                                            |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data        | A data frame. 1st column = DMU names.                                                                                                                                                                                      |
| inputs      | Column indices/names of input variables.                                                                                                                                                                                   |
| outputs     | Column indices/names of output variables.                                                                                                                                                                                  |
| ud_outputs  | Column indices (within outputs) of undesirable outputs, or NULL.                                                                                                                                                           |
| orientation | "io" (input-oriented, default) or "oo" (output-oriented). All DEA functions return Farrell efficiency (0~1), where 1 = efficient. For output orientation: value = $1/\phi$ , where $\phi \geq 1$ is the Shephard distance. |
| rts         | "vrs" (variable returns to scale, default) or "crs" (constant).                                                                                                                                                            |
| period      | Column name or index identifying the time period variable.                                                                                                                                                                 |
| type1       | "cont" (contemporaneous), "seq" (sequential), or "glob" (global, default).                                                                                                                                                 |
| type2       | "fgnz" (Färe-Grosskopf-Norris-Zhang) or "rd" (Ray-Desli, default).                                                                                                                                                         |

### Value

A list with components: efficiencies, lambdas, slacks, targets, returns, model, orientation, dm\_u.

### Functions

- `basic_DEA()`: Standard radial DEA (CCR/BCC)
- `super_DEA()`: Super-efficiency DEA (self excluded, radial only)
- `basic_SBM()`: Standard Slacks-Based Measure (SBM, Tone 2001)
- `super_SBM()`: Super-efficiency SBM (self excluded, no undesirable outputs)
- `malmquist()`: Malmquist productivity index

### Examples

```

df = data.frame(
  DMU = paste0("DMU", 1:7),
  x1 = c(20, 60, 40, 60, 70, 30, 50),
  x2 = c(151, 200, 120, 170, 250, 210, 90),
  y1 = c(100, 210, 150, 240, 220, 80, 200)
)
basic_DEA(df, inputs = 2:3, outputs = 4, rts = "crs")
basic_DEA(df, inputs = 2:3, outputs = 4, rts = "vrs")
df = data.frame(
  DMU = paste0("DMU", 1:7),
  x1 = c(20, 60, 40, 60, 70, 30, 50),
  x2 = c(151, 200, 120, 170, 250, 210, 90),
  y1 = c(100, 210, 150, 240, 220, 80, 200)
)
super_DEA(df, inputs = 2:3, outputs = 4, rts = "crs")

```

```

df = data.frame(
  DMU = paste0("DMU", 1:7),
  x1 = c(20, 60, 40, 60, 70, 30, 50),
  x2 = c(151, 200, 120, 170, 250, 210, 90),
  y1 = c(100, 210, 150, 240, 220, 80, 200)
)
basic_SBM(df, inputs = 2:3, outputs = 4, rts = "crs")
df = data.frame(
  DMU = paste0("DMU", 1:7),
  x1 = c(20, 60, 40, 60, 70, 30, 50),
  x2 = c(151, 200, 120, 170, 250, 210, 90),
  y1 = c(100, 210, 150, 240, 220, 80, 200)
)
super_SBM(df, inputs = 2:3, outputs = 4, rts = "crs")
panel = data.frame(
  DMU = rep(paste0("DMU", 1:5), 3),
  Period = rep(1:3, each = 5),
  x1 = c(10, 20, 15, 25, 30, 12, 22, 17, 27, 32, 14, 24, 19, 29, 34),
  y1 = c(100, 150, 120, 180, 200, 110, 160, 130, 190, 210, 120, 170, 140, 200, 220)
)
malmquist(panel, period = "Period", inputs = 3, outputs = 4,
  rts = "crs", type1 = "cont", type2 = "fgnz")

```

---

defuzzify

*Defuzzification Methods for Fuzzy Comprehensive Evaluation*


---

## Description

Implements defuzzification methods for fuzzy evaluation vectors, including weighted average and maximum membership methods.

## Usage

```
defuzzify(mu, scores, method = "weighted_average")
```

## Arguments

|        |                                                                                                                                   |
|--------|-----------------------------------------------------------------------------------------------------------------------------------|
| mu     | Numeric vector, membership degrees for evaluation levels, in $[0, 1]$ .                                                           |
| scores | Numeric vector, scores corresponding to each evaluation level (e.g., c(100, 80, 60, 40) for "Excellent", "Good", "Fair", "Poor"). |
| method | Character, defuzzification method: "weighted_average", "max_membership", "centroid".                                              |

## Value

Numeric, defuzzified output value.

## Examples

```

# Example: Defuzzify fuzzy evaluation vectors for three schemes
mu = c(0.318, 0.351, 0.203, 0.128)
scores = c(30, 60, 75, 90) # Scores for "Poor", "Fair", "Good", "Excellent"
defuzzify(mu, scores, method = "weighted_average")
defuzzify(mu, scores, method = "max_membership")
defuzzify(mu, scores, method = "centroid")

```

---

|                |                              |
|----------------|------------------------------|
| entropy_weight | <i>Entropy Weight Method</i> |
|----------------|------------------------------|

---

## Description

Computes the weights of indicators and scores of samples based on the entropy method. This method objectively determines the importance of each indicator according to the amount of information it contains.

## Usage

```
entropy_weight(X, index = NULL, epsilon = 0.002)
```

## Arguments

|         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| X       | A numeric data frame or matrix where rows represent samples (observations) and columns represent indicators (variables).                                                                                                                                                                                                                                                                                                                                                                       |
| index   | A character vector indicating the direction of each indicator. Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better), and NA for already normalized indicators (no rescaling will be applied, but minor adjustments will still be made to avoid log(0) errors). If index = NULL (default), all indicators are treated as NA, meaning no normalization or rescaling is performed, but a small adjustment is still applied to prevent log(0) errors. |
| epsilon | A small constant used to replace exact 0s and 1s in the data to prevent log(0) errors. Default is 0.002.                                                                                                                                                                                                                                                                                                                                                                                       |

## Value

A list containing:

|   |                                                                |
|---|----------------------------------------------------------------|
| w | Numeric vector of weights for each indicator.                  |
| s | Numeric vector of scores for each sample (row), scaled by 100. |

## Examples

```
X = data.frame(
  x1 = c(3, 5, 2, 7),
  x2 = c(10, 20, 15, 25)
)
index = c("+", "-")
entropy_weight(X, index)
```

epi\_metrics

*Extract key epidemic metrics from ODE simulation output***Description**

This function computes core epidemiological summary statistics from deterministic compartmental epidemic models (e.g., SIR, SIS, SEIR).

**Usage**

```
epi_metrics(data, beta, gamma, N = NULL)
```

**Arguments**

|                    |                                                                                                                                                                                                                                     |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>data</code>  | A data frame returned by <code>model_*()</code> functions. Must contain a column <code>time</code> , and at least <code>S</code> and <code>I</code> . Additional compartments such as <code>E</code> or <code>R</code> are allowed. |
| <code>beta</code>  | Transmission rate (numeric).                                                                                                                                                                                                        |
| <code>gamma</code> | Recovery / removal rate (numeric).                                                                                                                                                                                                  |
| <code>N</code>     | Total population (numeric). If <code>NULL</code> (default), it is computed as the sum of all compartment values at the first time point.                                                                                            |

**Details**

The input data frame is expected to be the output of `model_*()` functions, containing a `time` column and one or more compartment columns (e.g., `S`, `I`, `R`, `E`). All compartment values are assumed to be in absolute population counts, and the total population is assumed to be conserved unless otherwise specified.

**Value**

A named list with:

**R0** Basic reproduction number.

**peak\_infection** Maximum number of infectious individuals.

**peak\_time** Time at which infectious peak occurs.

**attack\_rate** Proportion of population that transitioned out of `S`; defined only for models containing an `R` compartment (e.g., SIR/SEIR).

**Examples**

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
epi_metrics(sir, beta = 0.002, gamma = 0.1)
```

fuzzy\_eval

*Fuzzy Comprehensive Evaluation***Description**

Performs fuzzy comprehensive evaluation using different fuzzy composition operators to combine factor weights with a fuzzy evaluation matrix. Suitable for multi-criteria decision analysis with weights from methods like AHP, entropy, CRITIC, CV, or PCA.

**Usage**

```
fuzzy_eval(w, R, type)
```

**Arguments**

|      |                                                                                                                                                                                                                                                                                                                                                                           |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| w    | Numeric vector, factor weights (e.g., from <code>combine_weights</code> ).                                                                                                                                                                                                                                                                                                |
| R    | Numeric matrix, fuzzy evaluation matrix with columns as factors and rows as evaluation grades. Values should be in $[0, 1]$ .                                                                                                                                                                                                                                             |
| type | Integer or character (1-5), specifying the fuzzy composition operator: <ul style="list-style-type: none"> <li>• 1: Min-max (main factor decisive).</li> <li>• 2: Product-max (main factor prominent).</li> <li>• 3: Weighted sum (additive average).</li> <li>• 4: Bounded sum of mins (min-sum bounded).</li> <li>• 5: Normalized min-sum (balanced average).</li> </ul> |

**Details**

The function computes a fuzzy comprehensive evaluation vector  $B$  based on the weight vector  $w$  and fuzzy evaluation matrix  $R$ . Five composition operators are supported:

- Type 1 (min-max):  $\max(\min(w, R[j, ]))$ , emphasizes the main factor.
- Type 2 (product-max):  $\max(w * R[j, ])$ , highlights the main factor.
- Type 3 (weighted sum):  $\text{sum}(w * R[j, ])$ , additive average.
- Type 4 (bounded sum):  $\min(1, \text{sum}(\min(w, R[j, ])))$ , bounds the sum of mins.
- Type 5 (normalized min-sum):  $\text{sum}(\min(w, R[j, ] / \text{sum}(R[j, ])))$ , balanced average.

The output  $B$  is normalized to sum to 1. If the sum is zero, an error is thrown. Uses base R for lightweight implementation.

**Value**

A numeric vector of normalized comprehensive evaluation results, summing to 1.

**Examples**

```
w = c(0.3, 0.3, 0.3, 0.1) # weights (e.g., from AHP or entropy)

# fuzzy evaluation matrix (3 grades for 4 factors)
R = matrix(c(0.8, 0.7, 0.6, 0.7,
             0.1, 0.2, 0.2, 0.1,
             0.1, 0.1, 0.2, 0.2), nrow = 3, byrow = TRUE)
# Apply fuzzy comprehensive evaluation
fuzzy_eval(w, R, type = 3) # Weighted sum
```

GM11\_markov

*Grey-Markov Prediction Model***Description**

Fits a GM(1,1) model and then uses a Markov chain on relative error states to correct both historical fitted values and future predictions.

**Usage**

```
GM11_markov(X, n_ahead = 3, breaks = c(-0.1, -0.05, -0.02, 0, 0.02, 0.05, 0.1))
```

**Arguments**

|         |                                                                                                                           |
|---------|---------------------------------------------------------------------------------------------------------------------------|
| X       | Numeric vector of original time series data.                                                                              |
| n_ahead | Number of future periods to predict. Must be a positive integer.                                                          |
| breaks  | Numeric vector of boundaries for classifying relative errors into states. -Inf and Inf are prepended/appended internally. |

**Details**

The function first fits a GM(1,1) model to the original series, classifies relative errors into states using breaks, builds a Markov chain on these error states, and finally adjusts the GM(1,1) fitted and predicted values.

**Value**

A data frame with columns:

**Period** Time period labels, such as T1, T2, ..., T+n.

**Raw** Original values; future periods are NA.

**GM11\_fitted** GM(1,1) fitted or predicted values.

**err\_state** Relative error state for historical periods, and predicted state for future periods.

**adj\_eff** Adjustment effect, defined as the midpoint of the state interval.

**Markov\_adj** Markov-chain-adjusted fitted or predicted values.

**Examples**

```
X = c(174, 179, 183, 189, 207, 234, 220.5, 256, 270, 285)
GM11_markov(X, n_ahead = 3)
```

## Description

A collection of functions for performing grey relational analysis, including calculation of grey correlation degree and evaluation based on grey correlation. These functions are designed for decision-making and data analysis by measuring the relational degree between sequences.

## Usage

```
grey_corr(ref, cmp, rho = 0.5, w = NULL)
```

```
grey_corr_topsis(X, w, index = NULL, rho = 0.5)
```

## Arguments

|       |                                                                                                                                                                                                                                                                                                                            |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ref   | Numeric vector, the reference sequence for <code>grey_corr</code> .                                                                                                                                                                                                                                                        |
| cmp   | Numeric matrix or data frame, the comparison sequences for <code>grey_corr</code> .                                                                                                                                                                                                                                        |
| rho   | Numeric scalar, the distinguishing coefficient (default = 0.5).                                                                                                                                                                                                                                                            |
| w     | Numeric vector, weights for weighted correlation (default = equal weights).                                                                                                                                                                                                                                                |
| X     | Numeric matrix or data frame, the decision matrix for <code>grey_corr_topsis</code> .                                                                                                                                                                                                                                      |
| index | Character vector indicating indicator direction: Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better), and NA for already rescaled indicators (no rescaling will be applied). If index = NULL (default), all indicators are treated as NA, meaning no rescaling is performed. |

## Details

These functions implement grey relational analysis for evaluating relationships between sequences or decision alternatives:

**grey\_corr** Computes the grey correlation degree between a reference sequence (ref) and comparison sequences (cmp) using the distinguishing coefficient (rho) and optional weights (w).

**grey\_corr\_topsis** Evaluates a decision matrix (X) by normalizing it, applying weights (w), computing grey correlation with the ideal sequence. Direction of indicators can be specified via index.

## Value

**grey\_corr** Returns a numeric vector of grey correlation degrees for each comparison sequence.

**grey\_corr\_topsis** Returns a numeric vector of relative closeness (grey correlation degrees).

## Examples

```
# Grey correlation degree
ref = c(0.9, 0.8, 0.7)
cmp = data.frame(
  x1 = c(0.9, 0.7, 0.8),
  x2 = c(0.8, 0.9, 0.7),
```

```

    x3 = c(0.7, 0.8, 0.9)
  )
  grey_corr(ref, cmp, rho = 0.5)

  # Grey correlation evaluation
  X = data.frame(x1 = c(8, 7, 6), x2 = c(150, 180, 200), x3 = c(60, 80, 100))
  w = c(0.3, 0.4, 0.3)
  idx = c("+", "+", "+")
  grey_corr_topsis(X, w, idx, rho = 0.5)

```

grey\_models

*Grey Prediction Models***Description**

Implements grey prediction models for time series forecasting: GM11 applies the GM(1,1) model with level ratio test. GM1N applies the GM(1,N) model with multiple related factors. DGM21 applies the DGM(2,1) model for second-order dynamics. verhulst applies the Verhulst model for logistic growth.

**Usage**

```
GM11(X)
```

```
GM1N(dat, new_data = NULL)
```

```
DGM21(X)
```

```
verhulst(X)
```

**Arguments**

|          |                                                                                                   |
|----------|---------------------------------------------------------------------------------------------------|
| X        | For GM11, DGM21, verhulst: Numeric vector of original time series data.                           |
| dat      | For GM1N: Data frame or matrix, last column is characteristic series, others are related factors. |
| new_data | For GM1N: Optional future values of related factors (1 row, m-1 columns).                         |

**Value**

For GM11: List with fitted values (`fitted`), next prediction (`pnext`), prediction function (`f`), matrix (`mat`), parameters (`u`), level ratios (`lambda`), and range (`rng`). For GM1N: List with fitted values (`fitted`), posterior variance ratio (`C`), small error probability (`P`), and prediction function (`f`). For DGM21, verhulst: List with fitted values (`fitted`), next prediction (`pnext`), prediction function (`f`), matrix (`mat`), and parameters (`u`).

**Examples**

```

# Sample time series for GM11, DGM21, Verhulst
x = c(100, 120, 145, 175, 210)

# GM11

```



```

result = GM11(x)
result$fitted      # Fitted values
result$pnext       # Next prediction
result$f(6:8)      # Predict next 3 periods

# DGM21
x = c(2.874,3.278,3.39,3.679,3.77,3.8)
result = DGM21(x)
result$fitted      # Fitted values
result$pnext       # Next prediction
result$f(6:8)      # Predict next 3 periods

# Verhulst
x = c(4.93,2.33,3.87,4.35,6.63,7.15,5.37,6.39,7.81,8.35)
result = verhulst(x)
result$fitted      # Fitted values
result$pnext       # Next prediction
result$f(6:8)      # Predict next 3 periods

# Sample data for GM1N
data = data.frame(
  factor1 = c(50, 55, 60, 65, 70),
  factor2 = c(20, 22, 25, 28, 30),
  output = c(100, 120, 145, 175, 210)
)
result = GM1N(data)
result$fitted

```

---

inequality

*Inequality Indices*


---

## Description

Computes inequality indices for individual or grouped data: `gini0` calculates the Gini coefficient for individual sample data. `gini` calculates the Gini coefficient for grouped data using income and population shares. `theil0` calculates the Theil index for individual sample data. `theil` calculates the Theil index for grouped average data. `theil0_g` calculates the Theil index and decomposition for grouped sample data. `theil_g` calculates the Theil index and decomposition for grouped average data. `theil_g2_cross` calculates the Theil index and decomposition for two-level cross-grouped average data. `theil_g2_nest` calculates the Theil index and decomposition for two-level nested grouped average data.

For `theil`, `theil_g`, `theil_g2_cross`, `theil_g2_nest`: Name of population variable (character).

## Usage

```
gini0(y)
```

```
gini(y, pop)
```

```
theil0(y)
```

```
theil(y, pop)
```

```

theil0_g(data, group, y)

theil_g(data, group, y, pop)

theil_g2_cross(data, group1, group2, y, pop)

theil_g2_nest(data, group1, group2, y, pop)

```

### Arguments

|                     |                                                                                                                                                                                                                                                           |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>y</code>      | For <code>gini0</code> , <code>gini</code> , <code>theil0</code> : Numeric vector of individual incomes.                                                                                                                                                  |
| <code>pop</code>    | For <code>gini</code> : Numeric vector of group populations or population shares. For <code>theil</code> , <code>theil0_g</code> , <code>theil_g</code> , <code>theil_g2_cross</code> , <code>theil_g2_nest</code> : Name of income variable (character). |
| <code>data</code>   | For <code>theil0_g</code> , <code>theil_g</code> , <code>theil_g2_cross</code> , <code>theil_g2_nest</code> : Data frame containing variables.                                                                                                            |
| <code>group</code>  | For <code>theil0_g</code> , <code>theil_g</code> : Name of grouping variable (e.g., province).                                                                                                                                                            |
| <code>group1</code> | For <code>theil_g2_cross</code> , <code>theil_g2_nest</code> : Name of first grouping variable (e.g., region or province).                                                                                                                                |
| <code>group2</code> | For <code>theil_g2_cross</code> , <code>theil_g2_nest</code> : Name of second grouping variable (e.g., type or city).                                                                                                                                     |

### Value

For `gini0`, `gini`: Numeric Gini coefficient (0 to 1). For `theil0`, `theil`: Numeric Theil index. For `theil0_g`, `theil_g`, `theil_g2_cross`, `theil_g2_nest`: List with two vectors:

- `theil`: `theil` (Theil index and its decomposition),
- `ratio`: ratio (contribution rates of each component).

### Examples

```

# Sample data
income = c(10, 20, 30, 40, 100)
pop = c(100, 150, 200, 250, 300)

# Gini coefficient (individual data)
gini0(income)

# Gini coefficient (grouped data)
gini(income, pop)

# Theil index (individual sample)
data = data.frame(g = c("A", "A", rep("B", 10), rep("A", 6)),
                  y = c(10, 10, rep(8, 4), rep(6, 6), rep(4, 4), 2, 2))
theil0(data$y)

# Theil index (grouped average)
data2 = data |> dplyr::count(g, y, name = "pop")
theil(data2$y, data2$pop)

# Theil index with grouping (sample data)

```

```

theil0_g(data, "g", "y")

# Theil index with grouping (average data)
theil_g(data2, "g", "y", "pop")

# Theil index with two-level cross-grouping
data3 = data.frame(
  industry = c("A", "A", "A", "A", "B", "B", "B", "B"),
  area = c("East", "East", "West", "West", "East", "East", "West", "West"),
  province = c("Shanghai", "Beijing", "Sichuan", "Yunnan",
               "Shanghai", "Beijing", "Sichuan", "Yunnan"),
  avg_wage = c(500, 400, 80, 60, 300, 250, 50, 40),
  emp_num = c(100, 80, 90, 70, 120, 100, 80, 60)
)
theil_g2_cross(data3, "industry", "area", "avg_wage", "emp_num")

# Theil index with two-level nested grouping
data4 = data.frame(
  province = c("A", "A", "A", "A", "B", "B"),
  city = c("A1", "A1", "A2", "A2", "B1", "B1"),
  industry = c("Manu", "Serv", "Manu", "Serv", "Manu", "Serv"),
  y = c(50000, 45000, 60000, 55000, 70000, 65000),
  pop = c(10000, 8000, 15000, 12000, 10000, 8000)
)
theil_g2_nest(data4, "province", "city", "y", "pop")

```

linear\_sum

*Linear weighted synthesis***Description**

Linear weighted synthesis

**Usage**

```
linear_sum(data, w)
```

**Arguments**

|      |                                                                       |
|------|-----------------------------------------------------------------------|
| data | Indicator matrix or data frame (rows = samples, columns = indicators) |
| w    | Weight vector (length must match ncol(data))                          |

**Value**

Vector of comprehensive scores

**Examples**

```

data = data.frame(
  GDP = c(0.85, 0.72, 0.91, NA),
  Employment = c(0.78, 0.85, 0.67, 0.73),
  Environment = c(0.65, 0.72, NA, 0.81)
)

```

```
w = c(0.5, 0.3, 0.2)
linear_sum(data, w)
```

---

markov\_chain

*Markov Chain Prediction*


---

## Description

Constructs a transition probability matrix from a state sequence and performs multi-step Markov chain prediction.

## Usage

```
markov_chain(S, s0, n_steps = 5)
```

## Arguments

|                      |                                                                    |
|----------------------|--------------------------------------------------------------------|
| <code>S</code>       | State sequence, supplied as a vector or factor.                    |
| <code>s0</code>      | Initial state, must be one of the state levels in <code>S</code> . |
| <code>n_steps</code> | Number of prediction steps. Must be a positive integer.            |

## Details

For states that never appear as a starting state, equal transition probabilities are assigned to keep each row sum equal to 1. The ultimate stationary distribution is computed using eigenvalue decomposition.

## Value

A list with components:

**trans\_mat** Transition probability matrix.  
**pred\_probs** Matrix of state probabilities for each prediction step.  
**pred\_states** Predicted states, taking the most likely state at each step.  
**pi\_final** Ultimate stationary distribution.

## Examples

```
# Weather states: Rainy, Cloudy, Sunny
S = factor(c("Sunny", "Sunny", "Cloudy", "Rainy", "Sunny",
             "Cloudy", "Sunny", "Sunny", "Rainy", "Cloudy",
             "Sunny", "Cloudy", "Rainy", "Sunny", "Sunny",
             "Cloudy", "Sunny", "Rainy", "Cloudy", "Sunny"),
           levels = c("Rainy", "Cloudy", "Sunny"))
markov_chain(S, s0 = "Cloudy", n_steps = 3)
```

## Description

A collection of functions to compute membership values for various fuzzy sets, including triangular, trapezoidal, Gaussian, generalized bell, two-parameter Gaussian, sigmoid, difference of sigmoids, product of sigmoids, Z-shaped, PI-shaped, and S-shaped membership functions. Includes a function to visualize membership functions using ggplot2. These are designed for evaluation models in mathematical modeling, compatible with `fuzzy_eval` in the `mathmodels` package.

## Usage

```
tri_mf(x, params)

trap_mf(x, params)

gauss_mf(x, params)

gbell_mf(x, params)

gauss2mf(x, params)

sigmoid_mf(x, params)

dsigmoid_mf(x, params)

psigmoid_mf(x, params)

z_mf(x, params)

pi_mf(x, params)

s_mf(x, params)

plot_mf(mf, xlim = c(0, 10), main = NULL)
```

## Arguments

- |        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| x      | Numeric vector, input values for which to compute membership.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| params | Numeric vector, parameters defining the membership function: <ul style="list-style-type: none"> <li>• For <code>tri_mf</code>: <math>c(a, b, c)</math>, where <math>a \leq b \leq c</math> (left base, peak, right base).</li> <li>• For <code>trap_mf</code>: <math>c(a, b, c, d)</math>, where <math>a \leq b \leq c \leq d</math> (left base, left top, right top, right base).</li> <li>• For <code>gauss_mf</code>: <math>c(\text{sigma}, c)</math>, where <math>\text{sigma} &gt; 0</math> (spread, center).</li> <li>• For <code>gbell_mf</code>: <math>c(a, b, c)</math>, where <math>a &gt; 0, b &gt; 0</math> (width, shape, center).</li> <li>• For <code>gauss2mf</code>: <math>c(s1, c1, s2, c2)</math>, where <math>s1 &gt; 0, s2 &gt; 0</math> (left spread, left center, right spread, right center).</li> <li>• For <code>sigmoid_mf</code>: <math>c(a, b)</math>, where <math>a &gt; 0</math> (slope, inflection point).</li> </ul> |

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                   | <ul style="list-style-type: none"> <li>• For <code>dsigmoid_mf</code>: <math>c(a_1, c_1, a_2, c_2)</math>, where <math>a_1 &gt; 0</math>, <math>a_2 &gt; 0</math> (slopes and inflection points for two sigmoids).</li> <li>• For <code>psigmoid_mf</code>: <math>c(a_1, c_1, a_2, c_2)</math>, where <math>a_1 &gt; 0</math>, <math>a_2 &gt; 0</math> (slopes and inflection points for two sigmoids).</li> <li>• For <code>z_mf</code>: <math>c(a, b)</math>, where <math>a &lt; b</math> (left base, right base).</li> <li>• For <code>pi_mf</code>: <math>c(a, b, c, d)</math>, where <math>a &lt; b &lt; c &lt; d</math> (left base, left shoulder, right shoulder, right base).</li> <li>• For <code>s_mf</code>: <math>c(a, b)</math>, where <math>a &lt; b</math> (left base, right base).</li> </ul> |
| <code>mf</code>   | Function, a membership function with fixed parameters (e.g., <code>function(x) tri_mf(x, c(2, 5, 8))</code> ).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <code>xlim</code> | Numeric vector of length 2, x-axis limits for plotting (default <code>c(0, 10)</code> ).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>main</code> | Character, plot title (default <code>NULL</code> , no title).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

## Details

These functions support evaluation models in mathematical modeling:

- `tri_mf`: Triangular membership, linear rise from  $a$  to  $b$  (peak) and fall to  $c$ .
- `trap_mf`: Trapezoidal membership, linear rise from  $a$  to  $b$ , plateau from  $b$  to  $c$ , fall to  $d$ .
- `gauss_mf`: Gaussian membership, bell-shaped curve centered at  $c$  with spread  $\sigma$ .
- `gbell_mf`: Generalized bell membership, bell-shaped curve with width  $a$ , shape  $b$ , and center  $c$ .
- `gauss2mf`: Two-parameter Gaussian membership, combining two Gaussians with spreads  $s_1$ ,  $s_2$  and centers  $c_1$ ,  $c_2$ .
- `sigmoid_mf`: Sigmoid membership, S-shaped curve with slope  $a$  and inflection point  $b$ .
- `dsigmoid_mf`: Difference of two sigmoids, combining slopes  $a_1$ ,  $a_2$  and inflection points  $c_1$ ,  $c_2$ .
- `psigmoid_mf`: Product of two sigmoids, combining slopes  $a_1$ ,  $a_2$  and inflection points  $c_1$ ,  $c_2$ .
- `z_mf`: Z-shaped membership, decreasing from 1 at  $a$  to 0 at  $b$ .
- `pi_mf`: PI-shaped membership, rising from  $a$  to  $b$ , plateau from  $b$  to  $c$ , falling to  $d$ .
- `s_mf`: S-shaped membership, increasing from 0 at  $a$  to 1 at  $b$ .
- `plot_mf`: Plots a membership function over `xlim` using `ggplot2`, suitable for tidyverse workflows.

Membership values can be used to construct fuzzy evaluation matrices for `fuzzy_eval`. Implemented in base R, except `plot_mf`, which requires `ggplot2`.

## Value

- For membership functions (`tri_mf`, `trap_mf`, `gauss_mf`, `gbell_mf`, `gauss2mf`, `sigmoid_mf`, `dsigmoid_mf`, `psigmoid_mf`, `z_mf`, `pi_mf`, `s_mf`): A numeric vector of membership values in  $[0, 1]$ , same length as  $x$ .
- For `plot_mf`: A `ggplot2` object, plotting the membership function.

**Examples**

```

# Define input values
x = 0:10

# Triangular membership
tri_mf(x, params = c(3, 6, 8))

# Trapezoidal membership
trap_mf(x, params = c(1, 5, 7, 8))

# Gaussian membership
gauss_mf(x, params = c(2, 5))

# Generalized bell membership
gbell_mf(x, params = c(2, 4, 6))

# Two-parameter Gaussian membership
gauss2mf(x, params = c(1, 3, 3, 4))

# Sigmoid membership
sigmoid_mf(x, params = c(2, 4))

# Difference of sigmoids membership
dsigmoid_y = dsigmoid_mf(x, params = c(5, 2, 5, 7))

# Product of sigmoids membership
psigmoid_mf(x, params = c(2, 3, -5, 8))

# Z-shaped membership
z_mf(x, params = c(3, 7))

# PI-shaped membership
pi_mf(x, params = c(1, 4, 5, 10))

# S-shaped membership
s_mf(x, params = c(1, 8))

## Not run:
# Visualize membership functions
plot_mf(\(x) tri_mf(x, c(3, 6, 8)), main = "Triangular MF")
plot_mf(\(x) trap_mf(x, c(1, 5, 7, 8)), main = "Trapezoidal MF")
plot_mf(\(x) gauss_mf(x, c(2, 5)), main = "Gaussian MF")
plot_mf(\(x) gbell_mf(x, c(2, 4, 6)), main = "Generalized Bell MF")
plot_mf(\(x) gauss2mf(x, c(1, 3, 3, 4)), main = "Two-Parameter Gaussian MF")
plot_mf(\(x) sigmoid_mf(x, c(2, 4)), main = "Sigmoid MF")
plot_mf(\(x) dsigmoid_mf(x, c(5, 2, 5, 7)), main = "Difference of Sigmoids MF")
plot_mf(\(x) psigmoid_mf(x, c(2, 3, -5, 8)), main = "Product of Sigmoids MF")
plot_mf(\(x) z_mf(x, c(3, 7)), main = "Z-Shaped MF")
plot_mf(\(x) pi_mf(x, c(1, 4, 5, 10)), main = "PI-Shaped MF")
plot_mf(\(x) s_mf(x, c(1, 8)), main = "S-Shaped MF")

## End(Not run)

```

model\_logistic

*Logistic Population Growth Model***Description**

Solves the logistic growth equation:

$$\frac{dN}{dt} = r \left( 1 - \frac{N}{K} \right) N$$

**Usage**

```
model_logistic(init, params, times, ...)
```

**Arguments**

|        |                                                                                                                                 |
|--------|---------------------------------------------------------------------------------------------------------------------------------|
| init   | Named numeric vector, e.g. <code>c(N = 10)</code> .                                                                             |
| params | Named numeric vector, e.g. <code>c(r = 0.5, K = 100)</code> .<br><b>r</b> Intrinsic growth rate.<br><b>K</b> Carrying capacity. |
| times  | Numeric vector of output times.                                                                                                 |
| ...    | Additional arguments passed to <code>ode_solver</code> .                                                                        |

**Details**

The analytical solution is  $N(t) = K / (1 + (K/N_0 - 1) e^{-rt})$ , which can be used to verify numerical accuracy.

**Value**

A `data.frame` with columns `time` and `N`.

**Examples**

```
res = model_logistic(
  init = c(N = 10),
  params = c(r = 0.5, K = 100),
  times = seq(0, 20, by = 0.1)
)
head(res)
```



model\_lv

*Lotka-Volterra Predator-Prey Model***Description**

Classic two-species predator-prey system:

$$\frac{dx}{dt} = \alpha x - \beta xy$$

$$\frac{dy}{dt} = \delta xy - \mu y$$

**Usage**

```
model_lv(init, params, times, ...)
```

**Arguments**

|                     |                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>init</code>   | Named numeric vector, e.g. <code>c(x = 40, y = 9)</code> .                                                                                                                                                                                                                                                                                                                            |
| <code>params</code> | Named numeric vector with four parameters:<br><b>alpha</b> Prey intrinsic growth rate.<br><b>beta</b> Predation rate (prey killed per predator per unit time).<br><b>delta</b> Predator reproduction rate per prey consumed.<br><b>mu</b> Predator mortality rate. Named <code>mu</code> (not <code>gamma</code> ) to avoid confusion with the recovery rate used in epidemic models. |
| <code>times</code>  | Numeric vector of output times.                                                                                                                                                                                                                                                                                                                                                       |
| <code>...</code>    | Additional arguments passed to <code>ode_solver</code> .                                                                                                                                                                                                                                                                                                                              |

**Details**

- `x` Prey population.
- `y` Predator population.

**Value**

A `data.frame` with columns `time`, `x`, `y`.

**Examples**

```
model_lv(
  init = c(x = 40, y = 9),
  params = c(alpha = 0.4, beta = 0.04, delta = 0.02, mu = 0.5),
  times = seq(0, 100, by = 0.1)
)
```

---

|               |                                              |
|---------------|----------------------------------------------|
| model_malthus | <i>Malthusian (Exponential) Growth Model</i> |
|---------------|----------------------------------------------|

---

## Description

Solves the Malthusian growth equation:

$$\frac{dN}{dt} = r N$$

## Usage

```
model_malthus(init, params, times, ...)
```

## Arguments

|        |                                                                                                         |
|--------|---------------------------------------------------------------------------------------------------------|
| init   | Named numeric vector, e.g. <code>c(N = 100)</code> .                                                    |
| params | Named numeric vector, e.g. <code>c(r = 0.3)</code> .<br><b>r</b> Intrinsic growth rate (per time unit). |
| times  | Numeric vector of output times.                                                                         |
| ...    | Additional arguments passed to <code>ode_solver</code> .                                                |

## Details

The analytical solution is  $N(t) = N_0 e^{rt}$ , which can be used to verify numerical accuracy.

## Value

A `data.frame` with columns `time` and `N`.

## Examples

```
res = model_malthus(
  init = c(N = 100),
  params = c(r = 0.3),
  times = seq(0, 10, by = 0.1)
)
# Compare with analytical solution
res$N_exact = 100 * exp(0.3 * res$time)
head(res)
```

model\_seir

*SEIR Epidemic Model***Description**

Four-compartment model that adds an exposed (latent) class:

$$\begin{aligned}\frac{dS}{dt} &= -\beta SI \\ \frac{dE}{dt} &= \beta SI - \alpha E \\ \frac{dI}{dt} &= \alpha E - \gamma I \\ \frac{dR}{dt} &= \gamma I\end{aligned}$$

**Usage**

```
model_seir(init, params, times, ...)
```

**Arguments**

|                     |                                                                                                                                                                                                                                                                                              |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>init</code>   | Named numeric vector. <code>R</code> defaults to 0 if omitted, e.g. <code>c(S = 980, E = 10, I = 10)</code> .                                                                                                                                                                                |
| <code>params</code> | Named numeric vector, e.g. <code>c(beta = 0.3, alpha = 0.2, gamma = 0.1)</code> .<br><b>beta</b> Transmission rate.<br><b>alpha</b> Rate of progression from exposed to infectious ( $1/\alpha$ = mean latent period).<br><b>gamma</b> Recovery rate ( $1/\gamma$ = mean infectious period). |
| <code>times</code>  | Numeric vector of output times.                                                                                                                                                                                                                                                              |
| <code>...</code>    | Additional arguments passed to <code>ode_solver</code> .                                                                                                                                                                                                                                     |

**Details**

- **S** Susceptible — not yet infected.
- **E** Exposed — infected but not yet infectious (mean latent period  $1/\alpha$ ).
- **I** Infectious — actively spreading the pathogen (mean infectious period  $1/\gamma$ ).
- **R** Recovered — permanently immune.

Population size  $N = S + E + I + R$  is conserved.

**Value**

A `data.frame` with columns `time`, `S`, `E`, `I`, `R`.

**Examples**

```
model_seir(
  init = c(S = 980, E = 10, I = 10),
  params = c(beta = 0.3, alpha = 0.2, gamma = 0.1),
  times = seq(0, 50, by = 0.1)
)
```

---

|          |                          |
|----------|--------------------------|
| model_si | <i>SI Epidemic Model</i> |
|----------|--------------------------|

---

## Description

Two-compartment model with no recovery:

$$\frac{dS}{dt} = -\beta SI$$

$$\frac{dI}{dt} = \beta SI$$

## Usage

```
model_si(init, params, times, ...)
```

## Arguments

|                     |                                                                   |
|---------------------|-------------------------------------------------------------------|
| <code>init</code>   | Named numeric vector, e.g. <code>c(S = 990, I = 10)</code> .      |
| <code>params</code> | Named numeric vector, e.g. <code>c(beta = 0.002)</code> .         |
|                     | <b>beta</b> Transmission rate (contacts per individual per time). |
| <code>times</code>  | Numeric vector of output times.                                   |
| <code>...</code>    | Additional arguments passed to <code>ode_solver</code> .          |

## Details

Population size  $N = S + I$  is conserved.

## Value

A `data.frame` with columns `time`, `S`, `I`.

## Examples

```
model_si(
  init = c(S = 990, I = 10),
  params = c(beta = 0.002),
  times = seq(0, 30, by = 0.1)
)
```

model\_sir

*SIR Epidemic Model***Description**

Three-compartment model with permanent immunity after recovery:

$$\frac{dS}{dt} = -\beta SI$$

$$\frac{dI}{dt} = \beta SI - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

**Usage**

```
model_sir(init, params, times, ...)
```

**Arguments**

|                     |                                                                                                                                                                                              |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>init</code>   | Named numeric vector. R defaults to 0 if omitted, e.g. <code>c(S = 990, I = 10)</code> .                                                                                                     |
| <code>params</code> | Named numeric vector, e.g. <code>c(beta = 0.002, gamma = 0.1)</code> .<br><b>beta</b> Transmission rate.<br><b>gamma</b> Recovery rate ( $1/\text{gamma} = \text{mean infectious period}$ ). |
| <code>times</code>  | Numeric vector of output times.                                                                                                                                                              |
| <code>...</code>    | Additional arguments passed to <code>ode_solver</code> .                                                                                                                                     |

**Details**

Population size  $N = S + I + R$  is conserved. The basic reproduction number is  $R_0 = \beta N / \gamma$ .

**Value**

A `data.frame` with columns `time`, `S`, `I`, `R`.

**Examples**

```
model_sir(
  init = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times = seq(0, 50, by = 0.1)
)
```

---

model\_sis

*SIS Epidemic Model*


---

## Description

Two-compartment model with recovery but no lasting immunity:

$$\frac{dS}{dt} = -\beta SI + \gamma I$$

$$\frac{dI}{dt} = \beta SI - \gamma I$$

## Usage

```
model_sis(init, params, times, ...)
```

## Arguments

|                     |                                                                                                                                                                                      |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>init</code>   | Named numeric vector, e.g. <code>c(S = 990, I = 10)</code> .                                                                                                                         |
| <code>params</code> | Named numeric vector, e.g. <code>c(beta = 0.002, gamma = 0.1)</code> .<br><b>beta</b> Transmission rate.<br><b>gamma</b> Recovery rate ( $1/\text{gamma}$ = mean infectious period). |
| <code>times</code>  | Numeric vector of output times.                                                                                                                                                      |
| <code>...</code>    | Additional arguments passed to <code>ode_solver</code> .                                                                                                                             |

## Details

Population size  $N = S + I$  is conserved.

## Value

A `data.frame` with columns `time`, `S`, `I`.

## Examples

```
model_sis(
  init = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times = seq(0, 50, by = 0.1)
)
```

ode\_solver

*General ODE Solver***Description**

Solves a system of ordinary differential equations (ODEs) using string-formula equations. Equation strings are parsed once at call time and reused across all integration steps, so the solver is substantially faster than approaches that call `parse()` inside the derivative function.

**Usage**

```
ode_solver(init, times, equations, params = NULL, method = "lsoda", ...)
```

**Arguments**

|                        |                                                                                                                                |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>init</code>      | A named numeric vector of initial values for all state variables.                                                              |
| <code>times</code>     | A numeric vector of output times.                                                                                              |
| <code>equations</code> | A named character vector; names are variable names, values are derivative expressions (e.g. <code>c(S = "-beta*S*I")</code> ). |
| <code>params</code>    | A named numeric vector or list of parameters referenced inside equations.                                                      |
| <code>method</code>    | Integration method passed to <code>deSolve::ode</code> . Default <code>"lsoda"</code> .                                        |
| <code>...</code>       | Additional arguments forwarded to <code>deSolve::ode</code> .                                                                  |

**Details**

All equations are parsed with `parse()` exactly once before the integrator starts. A single pre-allocated environment is reused on every derivative evaluation, avoiding repeated memory allocation and garbage collection.

**Value**

A `data.frame` with a time column followed by one column per state variable.

**Examples**

```
# Built-in wrapper (one-liner)
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
head(sir)

# Custom SEIR model with demography
seir_demo = ode_solver(
  init = c(S = 1000, E = 1, I = 0, R = 0),
  times = seq(0, 200, by = 1),
  equations = c(
    S = "mu*N - beta*S*I - mu*S",
    E = "beta*S*I - alpha*E - mu*E",
    I = "alpha*E - gamma*I - mu*I",
  )
)
```

```

      R = "gamma*I - mu*R"
    ),
    params = c(beta = 0.3, alpha = 0.2, gamma = 0.1, mu = 0.01, N = 1000)
  )
  tail(seir_demo)

```

pca\_weight

*PCA-Based Weighting Method*

## Description

Computes indicator weights using Principal Component Analysis (PCA). The method extracts principal components and uses their variance contribution to derive objective weights for indicators. Optionally handles positive/negative directions of indicators, and supports pre-standardized data.

## Usage

```
pca_weight(X, index = NULL, nfs = NULL, varimax = TRUE, method = "abs")
```

## Arguments

|         |                                                                                                                                                                                                                                                                                                                                                                     |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| X       | A numeric data frame or matrix where rows represent samples and columns represent indicators.                                                                                                                                                                                                                                                                       |
| index   | A character vector indicating the direction of each indicator. Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better), and NA for already standardized indicators (no standardization will be applied).<br><br>If `index = NULL` (default), all indicators are treated as `NA`, meaning no standardization is performed. |
| nfs     | Number of principal components to use; by default, all are used.                                                                                                                                                                                                                                                                                                    |
| varimax | Whether to perform Varimax rotation, default is TRUE.                                                                                                                                                                                                                                                                                                               |
| method  | Weighting Method. "abs" uses absolute loading values $ a_{ji} $ (default), "squared" uses $a_{ji}^2$ .                                                                                                                                                                                                                                                              |

## Value

A list containing:

|        |                                                           |
|--------|-----------------------------------------------------------|
| w      | Numeric vector of normalized weights for each indicator.  |
| s      | Numeric vector of scores for each sample.                 |
| lambda | Eigenvalues of principal components (explained variance). |
| varP   | Proportion of variance explained by selected PCs.         |

## Examples

```

# Example: Using PCA to compute indicator weights
ind = c("+", "+", "-", "-")
pca_weight(iris[1:10, 1:4], ind, nfs = 2)

```



---

|                   |                                      |
|-------------------|--------------------------------------|
| plot_compartments | <i>Plot compartment trajectories</i> |
|-------------------|--------------------------------------|

---

**Description**

Draws one line per compartment over time. Use compartments to select a subset of the state variables; the default is to plot all columns except time.

**Usage**

```
plot_compartments(df, compartments = NULL)
```

**Arguments**

|              |                                                                                                             |
|--------------|-------------------------------------------------------------------------------------------------------------|
| df           | A data frame returned by <code>ode_solver()</code> or any built-in model function.                          |
| compartments | Character vector of compartment names to plot. When NULL (the default) every column except time is plotted. |

**Value**

A [ggplot](#) object.

**Examples**

```
sir = model_sir(  
  init = c(S = 990, I = 10),  
  params = c(beta = 0.002, gamma = 0.1),  
  times = seq(0, 50, by = 0.1)  
)  
plot_compartments(sir)  
plot_compartments(sir, compartments = c("S", "I"))
```

---

|                |                                       |
|----------------|---------------------------------------|
| plot_incidence | <i>Plot daily new infections (dI)</i> |
|----------------|---------------------------------------|

---

**Description**

Computes the daily change in the infectious compartment  $\Delta I$  directly via `diff()` and plots it as a line chart. The peak is highlighted with a point marker.

**Usage**

```
plot_incidence(df)
```

**Arguments**

|    |                                                                                                                     |
|----|---------------------------------------------------------------------------------------------------------------------|
| df | A data frame returned by <code>ode_solver()</code> or any built-in model function. Must contain columns time and I. |
|----|---------------------------------------------------------------------------------------------------------------------|

**Value**

A [ggplot](#) object.

**Examples**

```
sir = model_sir(  
  init   = c(S = 990, I = 10),  
  params = c(beta = 0.002, gamma = 0.1),  
  times  = seq(0, 50, by = 0.1)  
)  
plot_incidence(sir)
```

---

|               |                          |
|---------------|--------------------------|
| plot_phase_si | <i>Phase plot S vs I</i> |
|---------------|--------------------------|

---

**Description**

Draws a trajectory in the  $(S, I)$  plane. Useful for visualising the epidemic orbit and threshold behaviour.

**Usage**

```
plot_phase_si(df)
```

**Arguments**

df                      A data frame with columns time, S, and I.

**Value**

A [ggplot](#) object.

**Examples**

```
sir = model_sir(  
  init   = c(S = 990, I = 10),  
  params = c(beta = 0.002, gamma = 0.1),  
  times  = seq(0, 50, by = 0.1)  
)  
plot_phase_si(sir)
```

---

|                  |                                                            |
|------------------|------------------------------------------------------------|
| plot_Rt_estimate | <i>Plot effective reproduction number <math>R_t</math></i> |
|------------------|------------------------------------------------------------|

---

### Description

Estimates the time-varying effective reproduction number  $R_t$  directly from compartment data. Two methods are available:

"mechanistic" Uses  $R_t = \beta S(t)/\gamma$ . This is the standard formula for an SIR-type model and is recommended when the model dynamics match the SIR structure.

"normalized" Uses  $R_t = R_0 S(t)/N$  with  $R_0 = \beta N/\gamma$ . Suitable when different normalisations are desired.

### Usage

```
plot_Rt_estimate(df, params, N = NULL, method = c("mechanistic", "normalized"))
```

### Arguments

|        |                                                                                                                         |
|--------|-------------------------------------------------------------------------------------------------------------------------|
| df     | A data frame with columns time and S.                                                                                   |
| params | A named numeric vector or list containing at least beta and gamma.                                                      |
| N      | Total population size. If NULL (the default), it is estimated from the sum of all state columns at the first time step. |
| method | Estimation method: "mechanistic" (default) or "normalized".                                                             |

### Value

A [ggplot](#) object with a dashed horizontal line at  $R_t = 1$ .

### Examples

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
plot_Rt_estimate(sir, params = c(beta = 0.002, gamma = 0.1))
```

---

|         |                           |
|---------|---------------------------|
| plot_ts | <i>Plot a Time Series</i> |
|---------|---------------------------|

---

### Description

Renders a clean line chart for one or more time series.

**Usage**

```
plot_ts(
  x,
  title = "Time Series",
  subtitle = NULL,
  x_lab = "Time",
  y_lab = "Value",
  colour = .PALETTE$observed,
  add_points = FALSE
)
```

**Arguments**

|                         |                                                                                                                         |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>x</code>          | A numeric vector, ts object, tidy tibble with columns (index, value), or a named list of such objects for multi-series. |
| <code>title</code>      | Character. Plot title.                                                                                                  |
| <code>subtitle</code>   | Character. Plot subtitle.                                                                                               |
| <code>x_lab</code>      | Character. x-axis label (default "Time").                                                                               |
| <code>y_lab</code>      | Character. y-axis label (default "Value").                                                                              |
| <code>colour</code>     | Character. Line colour (ignored for multi-series).                                                                      |
| <code>add_points</code> | Logical. Add point markers (default FALSE).                                                                             |

**Value**

A ggplot object.

**Examples**

```
data(AirPassengers)
plot_ts(AirPassengers, title = "Air Passengers", y_lab = "Thousands")
```

---

plot\_ts\_acf

---

ACF and PACF Plots

---

**Description**

ACF and PACF Plots

**Usage**

```
plot_ts_acf(
  x,
  max_lag = 40L,
  type = c("acf", "pacf", "both"),
  title = NULL,
  diff = 0L
)
```

**Arguments**

|                      |                                                                                 |
|----------------------|---------------------------------------------------------------------------------|
| <code>x</code>       | A numeric vector, ts object, or tidy tibble/data.frame with a value column.     |
| <code>max_lag</code> | Integer. Maximum lag (default 40).                                              |
| <code>type</code>    | Character. "acf" (default), "pacf", or "both" (returns a patchwork panel).      |
| <code>title</code>   | Character. Plot title.                                                          |
| <code>diff</code>    | Integer. Apply <code>diff()</code> this many times before plotting (default 0). |

**Value**

A ggplot or patchwork object.

**Examples**

```
data(AirPassengers)
plot_ts_acf(log(AirPassengers), type = "both")
```

---

|                               |                                                                |
|-------------------------------|----------------------------------------------------------------|
| <code>plot_ts_forecast</code> | <i>Plot Historical Series + Forecast with Confidence Bands</i> |
|-------------------------------|----------------------------------------------------------------|

---

**Description**

Works with the tibble returned by `ts_forecast()`.

**Usage**

```
plot_ts_forecast(
  observed,
  forecast_tbl,
  level = c(80, 95),
  title = "Forecast",
  y_lab = "Value",
  trim_n = NULL
)
```

**Arguments**

|                           |                                                                                                                   |
|---------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>observed</code>     | A numeric vector or ts object of the historical data.                                                             |
| <code>forecast_tbl</code> | Tibble from <code>ts_forecast()</code> .                                                                          |
| <code>level</code>        | Numeric vector. Which confidence levels to shade (must match columns in <code>forecast_tbl</code> ).              |
| <code>title</code>        | Character. Plot title.                                                                                            |
| <code>y_lab</code>        | Character. y-axis label.                                                                                          |
| <code>trim_n</code>       | Integer. Show only the last <code>trim_n</code> observations from history (default <code>NULL</code> = show all). |

**Value**

A ggplot object.

**Examples**

```
data(AirPassengers)
fit = ts_sarima(log(AirPassengers))
fc = ts_forecast(fit, h = 24)
plot_ts_forecast(log(AirPassengers), fc, title = "Log Air Passengers Forecast")
```

---

|               |                              |
|---------------|------------------------------|
| plot_ts_garch | <i>GARCH Volatility Plot</i> |
|---------------|------------------------------|

---

**Description**

Four-panel overview: (1) observed returns with  $\pm 2\sigma$  bands, (2) conditional variance, (3) standardised residuals, (4) histogram of standardised residuals.

**Usage**

```
plot_ts_garch(garch_result, title = "GARCH Volatility Analysis")
```

**Arguments**

|              |                              |
|--------------|------------------------------|
| garch_result | Result list from ts_garch(). |
| title        | Character. Plot title.       |

**Value**

A patchwork composite ggplot.

**Examples**

```
r = diff(log(AirPassengers))
res = ts_garch(r)
plot_ts_garch(res)
```

---

|              |                                      |
|--------------|--------------------------------------|
| plot_ts_pacf | <i>PACF Plot (convenience alias)</i> |
|--------------|--------------------------------------|

---

**Description**

Shortcut for plot\_ts\_acf(x, type = "pacf").

**Usage**

```
plot_ts_pacf(x, max_lag = 40L, title = NULL, diff = 0L)
```

**Arguments**

|         |                                                                             |
|---------|-----------------------------------------------------------------------------|
| x       | A numeric vector, ts object, or tidy tibble/data.frame with a value column. |
| max_lag | Integer. Maximum lag (default 40).                                          |
| title   | Character. Plot title.                                                      |
| diff    | Integer. Apply diff() this many times before plotting (default 0).          |

---

|                   |                                  |
|-------------------|----------------------------------|
| plot_ts_residuals | <i>Residual Diagnostic Plots</i> |
|-------------------|----------------------------------|

---

**Description**

Four-panel diagnostic: (1) residuals over time, (2) ACF of residuals, (3) Q-Q plot, (4) histogram vs normal density.

**Usage**

```
plot_ts_residuals(model_result, title = "Residual Diagnostics", max_lag = 30L)
```

**Arguments**

|              |                                                                                                                    |
|--------------|--------------------------------------------------------------------------------------------------------------------|
| model_result | Result from any <code>ts_*</code> () function that contains a <code>\$fitted</code> tibble with a residual column. |
| title        | Character. Plot title.                                                                                             |
| max_lag      | Integer. Maximum lag for ACF (default 30).                                                                         |

**Value**

A patchwork composite ggplot.

**Examples**

```
data(AirPassengers)
fit = ts_sarima(log(AirPassengers))
plot_ts_residuals(fit)
```

---

|                      |                                                                       |
|----------------------|-----------------------------------------------------------------------|
| plot_ts_sarima_garch | <i>Dual-Axis Plot for SARIMA-GARCH: Mean + Conditional Volatility</i> |
|----------------------|-----------------------------------------------------------------------|

---

**Description**

Upper panel shows observed values with mean fitted line; lower panel shows the conditional standard deviation from the GARCH component.

**Usage**

```
plot_ts_sarima_garch(
  sg_result,
  title = "SARIMA-GARCH: Mean & Volatility",
  y_lab = "Value"
)
```

**Arguments**

|           |                                                   |
|-----------|---------------------------------------------------|
| sg_result | Result list from <code>ts_sarima_garch()</code> . |
| title     | Character. Overall plot title.                    |
| y_lab     | Character. y-axis label for upper panel.          |

**Value**

A patchwork composite ggplot object.

**Examples**

```
data(AirPassengers)
res = ts_sarima_garch(log(AirPassengers))
plot_ts_sarima_garch(res)
```

---

|             |                                          |
|-------------|------------------------------------------|
| plot_ts_stl | <i>Plot STL Decomposition Components</i> |
|-------------|------------------------------------------|

---

**Description**

Stacked four-panel plot: observed, trend, seasonal, remainder.

**Usage**

```
plot_ts_stl(stl_result, title = "STL Decomposition")
```

**Arguments**

- stl\_result      Result list from ts\_stl().
- title           Character. Plot title.

**Value**

A patchwork composite ggplot.

**Examples**

```
data(AirPassengers)
res = ts_stl(AirPassengers)
plot_ts_stl(res)
```

---

|            |                                                                           |
|------------|---------------------------------------------------------------------------|
| preprocess | <i>Preprocessing Functions for Data Normalization and Standardization</i> |
|------------|---------------------------------------------------------------------------|

---

**Description**

A collection of functions to preprocess numeric data, including standardization, L2 norm normalization, Min-Max scaling, centered-type normalization, interval-type normalization, extreme-value-based normalization, initial-value-based normalization, mean-based normalization, and negative-to-positive transformation. These functions transform a numeric vector to a standardized or normalized scale, suitable for various indicator types (positive, negative, centered, interval-based, or extreme-based).



**Usage**

```
standardize(x, center = TRUE, scale = TRUE)
```

```
normalize(x)
```

```
rescale(x, type = "+", a = 0, b = 1)
```

```
rescale_middle(x, m)
```

```
rescale_interval(x, a, b)
```

```
rescale_extreme(x, type = "+")
```

```
rescale_initial(x, type = "+")
```

```
rescale_mean(x)
```

```
to_positive(x, type = "minmax")
```

**Arguments**

|        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| x      | Numeric vector to be preprocessed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| center | Logical or numeric scalar, passed to <code>base::scale</code> for centering (for <code>standardize</code> ). Default is <code>TRUE</code> .                                                                                                                                                                                                                                                                                                                                                                                                               |
| scale  | Logical or numeric scalar, passed to <code>base::scale</code> for scaling (for <code>standardize</code> ). Default is <code>TRUE</code> .                                                                                                                                                                                                                                                                                                                                                                                                                 |
| type   | Character scalar specifying the transformation direction or method:<br>" <b>+</b> " Positive direction (larger values are better, for <code>rescale</code> , <code>rescale_extreme</code> and <code>rescale_initial</code> ).<br>" <b>-</b> " Negative direction (smaller values are better, for <code>rescale</code> , <code>rescale_extreme</code> and <code>rescale_initial</code> ).<br>" <b>minmax</b> " Min-max transformation (for <code>to_positive</code> ).<br>" <b>reciprocal</b> " Reciprocal transformation (for <code>to_positive</code> ). |
| a      | Numeric scalar, lower bound of the output range or optimal interval (for <code>rescale</code> and <code>rescale_interval</code> ).                                                                                                                                                                                                                                                                                                                                                                                                                        |
| b      | Numeric scalar, upper bound of the output range or optimal interval (for <code>rescale</code> and <code>rescale_interval</code> ).                                                                                                                                                                                                                                                                                                                                                                                                                        |
| m      | Numeric scalar, optimal value for centered-type normalization (for <code>rescale_middle</code> ).                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

**Details**

These functions support various preprocessing needs in data analysis:

- `standardize`: Applies Z-score standardization (mean = 0, sd = 1), ideal for equalizing variances or normally distributed data.
- `normalize`: Scales the vector to unit length by dividing by its L2 (Euclidean) norm, useful for machine learning or similarity calculations.
- `rescale`: Performs Min-Max scaling to a specified range (default `[0, 1]`), supporting positive or negative indicators.

- `rescale_middle`: Normalizes centered-type indicators, where values closer to an optimal value  $m$  are better, mapping to  $[0, 1]$ .
- `rescale_interval`: Normalizes interval-type indicators, where values within  $[a, b]$  are optimal, mapping to  $[0, 1]$ .
- `rescale_extreme`: Normalizes using extreme values:  $\min(x)/x$  for positive indicators or  $x/\max(x)$  for negative indicators, often used in grey relational analysis.
- `rescale_initial`: Normalizes by dividing by the first value ( $x/x[1]$  or  $x[1]/x$ ), commonly used in grey relational analysis.
- `rescale_mean`: Normalizes by dividing by the mean ( $x/\text{mean}(x)$ ), commonly used in grey relational analysis.
- `to_positive`: Converts negative indicators to positive using either min-max ( $\max(x) - x$ ) or reciprocal ( $1/x$ ) transformation.

Missing values (NA) are preserved in the output. For `rescale_initial` and `rescale_mean`, the initial value or mean must be non-zero, respectively.

## Value

A numeric vector of the same length as `x`, transformed as follows:

- `standardize`: Standardized values (mean = 0, sd = 1).
- `normalize`: L2 norm normalized values (Euclidean norm, unit length).
- `rescale`: Min-Max scaled values in  $[a, b]$  (default  $[0, 1]$ ).
- `rescale_middle`: Centered-type normalized values in  $[0, 1]$ , where 1 indicates  $x = m$ .
- `rescale_interval`: Interval-type normalized values in  $[0, 1]$ , where 1 indicates  $x$  in  $[a, b]$ .
- `rescale_extreme`: Extreme-based normalized values using  $\min(x)/x$  (positive) or  $x/\max(x)$  (negative).
- `rescale_initial`: Initial-based normalized values using  $x/x[1]$  or  $x[1]/x$ .
- `rescale_mean`: Mean-based normalized values using  $x/\text{mean}(x)$ .
- `to_positive`: Transformed values converting negative indicators to positive using min-max or reciprocal transformation.

## Examples

```
# Standardization
x = c(4, 1, NA, 5, 8)
standardize(x)

# L2 norm normalization
normalize(x)

# Min-Max normalization (positive direction)
rescale(x) # Scale to [0, 1]
rescale(x, type = "-", a = 0.002, b = 0.996) # Reverse scaling

# Negative-to-positive transformation
to_positive(x) # Min-max transformation
to_positive(x, type = "reciprocal") # Reciprocal transformation

# Centered-type normalization
PH = 6:9
```

```

rescale_middle(PH, 7)

# Interval-type normalization
Temp = c(35.2, 35.8, 36.6, 37.1, 37.8, 38.4)
rescale_interval(Temp, 36, 37)

# Extreme-based normalization
rescale_extreme(x)      # min(x)/x
rescale_extreme(x, "-") # x/max(x)

# Initial-based normalization
rescale_initial(x)

# Mean-based normalization
rescale_mean(x)

```

---

|                |                                        |
|----------------|----------------------------------------|
| rank_sum_ratio | <i>Rank Sum Ratio (RSR) Evaluation</i> |
|----------------|----------------------------------------|

---

### Description

Performs Rank Sum Ratio (RSR) evaluation on a dataset of positive indicators, computing ranks, weighted RSR values, and a linear regression model to fit RSR against probit-transformed ranks. Supports integer or non-integer ranking methods.

### Usage

```
rank_sum_ratio(data, w = NULL, method = "int")
```

### Arguments

|        |                                                                                                                      |
|--------|----------------------------------------------------------------------------------------------------------------------|
| data   | Data frame with positive indicator data; first column is an ID column for identifying evaluation objects.            |
| w      | Numeric vector, weights for indicators (default = equal weights).                                                    |
| method | Character scalar, ranking method: "int" for integer ranks or "non-int" for scaled ranks in [1, n] (default = "int"). |

### Details

The `rank_sum_ratio` function implements the RSR method for evaluating objects based on positive indicators. It ranks the indicators (using integer or non-integer methods), computes weighted RSR values, adjusts ranks with probit transformation, and fits a linear regression model to relate RSR to probit values. The function assumes the first column of data is an ID column, and weights (*w*) can be provided or set to equal weights by default.

### Value

A list containing:

- `resultTable`: Data frame with RSR values, ranks, cumulative frequencies, probit values, and fitted RSR values.
- `reg`: Linear model object fitting RSR against probit values.
- `rankTable`: Data frame with ranked indicator values.

**Examples**

```
# Example data
data = data.frame(ID = c("A", "B", "C"), X1 = c(10, 20, 15), X2 = c(5, 10, 8))
w = c(0.4, 0.6)
rank_sum_ratio(data, w, method = "int")
```

read\_nbs

*Read and Combine National Bureau of Statistics XLS Files***Description**

This function reads multiple XLS files downloaded from the National Bureau of Statistics of China website (<http://www.stats.gov.cn>) without requiring renaming. It extracts the variable name from the "indicator: XXX" text in cell A1 of each file, skips the first 3 rows of headers, reads the first 31 rows of data, and combines the data into a single data frame by stacking vertically based on the indicator names. It also simplifies region names by removing suffixes such as "city", "province", "autonomous region", "clan", or "Uyghur".

**Usage**

```
read_nbs(paths)
```

**Arguments**

paths                      A character vector containing the file paths to the XLS files.

**Value**

A tibble containing the combined data with an additional column indicator representing the original indicator names and a region column with simplified region names.

**Examples**

```
## Not run:
paths = c("file1.xls", "file2.xls")
data = read_nbs(paths)

## End(Not run)
```

regional\_economics

*Regional Economics Functions***Description**

A collection of functions for calculating regional economics indices, including Location Quotient (LQ), Herfindahl-Hirschman Index (HHI), and Ellison-Glaeser Index (EG). These functions are designed for analyzing regional and industrial data to assess spatial concentration and market structure.

**Usage**

```
LQ(data, region, total, .cols, .by = NULL)
```

```
HHI(x, scaled = FALSE)
```

```
EG(data, region, industry, y)
```

**Arguments**

|                       |                                                                                                                                                                                                                                                                                                                       |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>data</code>     | A data frame containing the necessary data for calculations. For LQ, the first row is assumed to be the national total, with the first two columns specifying the region and total, optionally including a grouping column. For EG, it contains region, industry, and indicator columns (e.g., employment or output). |
| <code>region</code>   | The name of the column in data specifying the region (used in LQ and EG).                                                                                                                                                                                                                                             |
| <code>total</code>    | The name of the column in data specifying the total (used in LQ).                                                                                                                                                                                                                                                     |
| <code>.cols</code>    | The columns in data for which to calculate the location quotient (used in LQ).                                                                                                                                                                                                                                        |
| <code>.by</code>      | Optional grouping column for LQ, defaults to NULL (no grouping).                                                                                                                                                                                                                                                      |
| <code>x</code>        | A numeric vector for calculating the HHI (used in HHI).                                                                                                                                                                                                                                                               |
| <code>scaled</code>   | Logical; if TRUE, the HHI is scaled to account for the number of firms. Defaults to FALSE.                                                                                                                                                                                                                            |
| <code>industry</code> | The name of the column in data specifying the industry (used in EG).                                                                                                                                                                                                                                                  |
| <code>y</code>        | The name of the column in data specifying the indicator (e.g., employment or output, used in EG).                                                                                                                                                                                                                     |

**Details**

**LQ** Calculates the Location Quotient for multiple columns with optional grouping. The LQ measures the relative concentration of an industry in a region compared to a national benchmark. The function assumes the first row of data contains national totals, with the first two columns specifying the region and total, and the remaining columns used for LQ calculation.

**HHI** Calculates the Herfindahl-Hirschman Index, a measure of market concentration based on the squared sum of market shares. If `scaled = TRUE`, the HHI is normalized to account for the number of firms.

**EG** Calculates the Ellison-Glaeser Index, which measures the geographic concentration of an industry while controlling for firm size distribution and random distribution effects. It requires data on regions, industries, and an indicator (e.g., employment or output).

**Value**

**LQ** A data frame with the region column and calculated location quotients for the specified columns, optionally grouped by `.by`.

**HHI** A numeric value representing the HHI, either scaled or unscaled.

**EG** A tibble with two columns: the industry name and the corresponding EG index value.

**Examples**

```
# Example data
data = data.frame(
  region = c("National", "Region_A", "Region_B"),
```

```

    total = c(10000, 4000, 6000),
    industry1 = c(2000, 1000, 1000),
    industry2 = c(6000, 2000, 4000)
  )

  # Calculate Location Quotient
  LQ(data, region, total, starts_with("industry"))

  # Calculate HHI
  x = c(50, 30, 20)
  # Calculate the raw HHI
  HHI(x)
  # Calculate the standard (scaled) HHI
  HHI(x, scaled = TRUE)

  # Example data for EG
  eg_data = data.frame(
    region = c("R1", "R1", "R1", "R1", "R2", "R2", "R3", "R3", "R1", "R2", "R2", "R3"),
    industry = c("A", "A", "A", "A", "A", "A", "A", "A", "B", "B", "B", "B"),
    employment = c(250, 200, 150, 100, 20, 15, 10, 5, 50, 200, 150, 50)
  )
  EG(eg_data, region, industry, y = employment)

```

---

system\_evaluation

System Evaluation Functions for Coupling and Obstacle Analysis

---

## Description

These functions provide tools for system-level evaluation in multi-indicator systems:

- `coupling_degree()`: Computes coupling degree, coordination index, and coupling coordination degree for subsystems.
- `obstacle_degree()`: Computes obstacle degrees for secondary indicators to identify key constraints in the system, enabling batch processing with tidyverse for grouping and summarization.

## Usage

```
coupling_degree(data, w = NULL, id_cols = NULL, type = "standard")
```

```
obstacle_degree(data, w = NULL, id_cols = NULL, scaled = FALSE)
```

## Arguments

|                      |                                                                                                          |
|----------------------|----------------------------------------------------------------------------------------------------------|
| <code>data</code>    | A data frame with normalized scores (usually in $[0, 1]$ ) as columns.                                   |
| <code>w</code>       | Optional vector of weights for indicators or subsystems; defaults to equal weights if NULL.              |
| <code>id_cols</code> | Optional character vector of column names in data to preserve as identifiers (not used in calculations). |

|        |                                                                                                                                                                                                                                                                                                                                                                              |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| type   | Either "standard" for the standard coupling formula (results concentrated near 1) or "adjusted" for the revised formula (results more uniformly distributed in $[0, 1]$ ), as proposed by Wang Shujia, Kong Wei, et al. in "Misconceptions and Corrections of Domestic Coupling Coordination Degree Models, Journal of Natural Resources, 2021, 36(3): 793–810 (In Chinese)" |
| scaled | Logical. Whether to perform row normalization on obstacle degrees (default FALSE).                                                                                                                                                                                                                                                                                           |

### Value

A tibble depending on the function:

**coupling\_degree** A tibble with columns:

- ID: Identifier columns specified by `id_cols` (if provided).
- coupling: Coupling Degree (range 0-1).
- coord: Coordination Index (range 0-1).
- coupling\_coord: Coupling Coordination Degree (range 0-1).

**obstacle\_degree** A tibble with:

- ID: Identifier columns specified by `id_cols` (if provided).
- Columns for secondary indicator obstacle degrees ( $O_{ij} = (1 - X_{ij}) * w_{ij}$ ).

Suitable for grouping and summarizing (e.g., with tidyverse) to compute primary indicator obstacle degrees ( $\bigwedge U_i$ ).

### Examples

```
# Sample normalized subsystem scores
df = data.frame(
  ID = LETTERS[1:6],
  s1 = c(0.0162, 0.1782, 0.5490, 0.6730, 0.0207, 0.9875),
  s2 = c(0.2720, 0.6824, 0.0593, 0.4812, 0.8891, 0.5573),
  s3 = c(0.2655, 0.3721, 0.5729, 0.9082, 0.2017, 0.8984)
)
# Coupling Degree Analysis
coupling_degree(df, id_cols = "ID")          # Equal weights
coupling_degree(df, c(0.4, 0.3, 0.3), id_cols = "ID",
  type = "adjusted")                        # "adjusted" coupling degree
# Obstacle Degree Analysis
obstacle_degree(df, id_cols = "ID")          # Equal weights
obstacle_degree(df, c(0.4, 0.3, 0.3), id_cols = "ID")
```

### Description

Implements the Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS) to rank alternatives based on multiple criteria. The function normalizes the decision matrix using L2 norm, applies weights, and computes relative closeness to the ideal solution.

**Usage**

```
topsis(X, w = NULL, index = NULL)
```

**Arguments**

|                    |                                                                                                                                                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>X</code>     | A numeric matrix or data frame where rows represent alternatives and columns represent criteria.                                                                                                                                             |
| <code>w</code>     | A numeric vector of weights for each criterion. Must be non-negative and sum to 1. If not provided, equal weights are used.                                                                                                                  |
| <code>index</code> | A character vector indicating the direction of each indicator: Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better). If <code>index = NULL</code> (default), all indicators are treated as "+". |

**Details**

The TOPSIS method ranks alternatives by:

1. Normalizing the decision matrix using L2 norm normalization.
2. Applying weights to form a weighted normalized matrix.
3. Identifying positive and negative ideal solutions based on indicator directions.
4. Computing Euclidean distances to ideal solutions.
5. Calculating relative closeness as  $S_0 / (S_0 + S_{star})$ , where  $S_0$  is the distance to the negative ideal and  $S_{star}$  is the distance to the positive ideal.

This implementation supports both positive and negative indicators via the `index` parameter.

**Value**

A named numeric vector of relative closeness scores (in  $[0, 1]$ ) for each alternative. Higher values indicate better alternatives. Names are taken from `rownames(X)` or default to "Sample1", "Sample2", etc.

**Examples**

```
A = data.frame(
  X1 = c(2, 5, 3), # "+"
  X2 = c(8, 1, 6)  # "-"
)
w = c(0.6, 0.4)
idx = c("+", "-")
topsis(A, w, idx)
```



---

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| ts_back_transform | <i>Back-Transform Forecasts to the Original Scale</i> |
|-------------------|-------------------------------------------------------|

---

### Description

Reverses the operations applied by `ts_transform()` to restore point forecasts and confidence intervals to the original scale. Differencing is un-done by cumulative summation using the tail of the original series as the starting level.

### Usage

```
ts_back_transform(forecast_tbl, params, x_original)
```

### Arguments

|              |                                                                                                                     |
|--------------|---------------------------------------------------------------------------------------------------------------------|
| forecast_tbl | Tibble from <code>ts_forecast()</code> (columns: step, forecast, optionally lo_* and hi_*).                         |
| params       | The \$params element from a <code>ts_transform()</code> result.                                                     |
| x_original   | The original (pre-transform) ts or numeric vector, needed to supply the last observed level(s) for un-differencing. |

### Value

The forecast\_tbl with forecast and all interval columns converted to the original scale.

### Examples

```
data(AirPassengers)
tr = ts_transform(AirPassengers, method = "log", diff = 1,
                  seasonal_diff = 1, verbose = FALSE)
fit = ts_sarima(tr$transformed)
fc = ts_forecast(fit, h = 12)
fc_orig = ts_back_transform(fc, tr$params, AirPassengers)
```

---

|        |                                                              |
|--------|--------------------------------------------------------------|
| ts_ets | <i>ETS (Error, Trend, Seasonality) Exponential Smoothing</i> |
|--------|--------------------------------------------------------------|

---

### Description

Wraps `forecast::ets()` and returns a tidy result list.

### Usage

```
ts_ets(x, model = "ZZZ", frequency = NULL, ...)
```

**Arguments**

|                        |                                                                              |
|------------------------|------------------------------------------------------------------------------|
| <code>x</code>         | A numeric vector or ts object.                                               |
| <code>model</code>     | Character ETS model string, e.g. "ZZZ" (auto-select, default), "AAN", "AAA". |
| <code>frequency</code> | Integer. Required when <code>x</code> is a plain numeric vector.             |
| <code>...</code>       | Additional arguments forwarded to <code>forecast::ets()</code> .             |

**Value**

A named list:

**model\_info** One-row tibble: model type, AIC, AICc, BIC, log-likelihood.

**parameters** Tidy tibble of smoothing parameters and initial states.

**fitted** Tibble with index, observed, fitted, residual.

**model** Raw ets object.

**Examples**

```
data(AirPassengers)
res = ts_ets(AirPassengers)
res$model_info
```

---

ts\_forecast

---

*Generate Forecasts from a Fitted Time Series Model*


---

**Description**

A unified interface for forecasting from ETS, SARIMA, GARCH, or SARIMA-GARCH model results produced by the `ts_*`() functions.

**Usage**

```
ts_forecast(model_result, h = 12, level = c(80, 95), x_ts = NULL)
```

**Arguments**

|                           |                                                                                                                                     |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>model_result</code> | A result list from <code>ts_ets()</code> , <code>ts_sarima()</code> , <code>ts_garch()</code> , or <code>ts_sarima_garch()</code> . |
| <code>h</code>            | Integer. Forecast horizon (number of steps ahead).                                                                                  |
| <code>level</code>        | Numeric vector. Confidence levels in percent (default <code>c(80, 95)</code> ).                                                     |
| <code>x_ts</code>         | Original ts object (required for GARCH/SARIMA-GARCH to preserve time attributes). If <code>NULL</code> , integer steps are used.    |

**Value**

A tibble with columns:

**step** Forecast step (1 to `h`).

**forecast** Point forecast.

**lo\_** Lower bound for each confidence level.

**hi\_** Upper bound for each confidence level.

**sigma** Predicted conditional std dev (GARCH models only).

## Examples

```
data(AirPassengers)
fit = ts_sarima(log(AirPassengers))
ts_forecast(fit, h = 24)
```

ts\_garch

*GARCH Variance Modeling*

## Description

Fits a GARCH(p,q) model (optionally with ARMA mean) using `rugarch::ugarchfit()` and returns tidy results.

## Usage

```
ts_garch(
  x,
  garch_order = c(1, 1),
  arma_order = c(0, 0),
  dist = "norm",
  frequency = NULL,
  ...
)
```

## Arguments

|                          |                                                                                                                        |
|--------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>x</code>           | A numeric vector or ts object (typically returns/residuals).                                                           |
| <code>garch_order</code> | Integer vector <code>c(p, q)</code> : GARCH order (default <code>c(1, 1)</code> ).                                     |
| <code>arma_order</code>  | Integer vector <code>c(p, q)</code> : ARMA order for the mean equation (default <code>c(0, 0)</code> = constant mean). |
| <code>dist</code>        | Character. Innovation distribution: "norm" (default), "std" (Student-t), "ged", "snorm", "sstd".                       |
| <code>frequency</code>   | Integer. Required when <code>x</code> is a plain vector.                                                               |
| <code>...</code>         | Additional arguments forwarded to <code>rugarch::ugarchspec()</code> .                                                 |

## Value

A named list:

**model\_info** One-row tibble: spec, distribution, log-likelihood, AIC, BIC.

**coefficients** Tidy coefficient tibble with estimates, std errors, t-stats, p-values.

**fitted** Tibble: index, observed, sigma, variance, std\_residual.

**diagnostics** ARCH-LM and Ljung-Box tests on squared residuals.

**model** Raw `uGARCHfit` object.

## Examples

```
set.seed(42)
r = diff(log(AirPassengers))
res = ts_garch(r)
res$model_info
```

---

ts\_sarima

---

*SARIMA Model Fitting*


---

## Description

Auto-selects or fits a user-specified SARIMA model via `forecast::auto.arima()` or `forecast::Arima()`, returning a comprehensive tidy result.

## Usage

```
ts_sarima(
  x,
  order = NULL,
  seasonal = NULL,
  frequency = NULL,
  stepwise = TRUE,
  approximation = TRUE,
  ...
)
```

## Arguments

|                            |                                                                                                           |
|----------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>x</code>             | A numeric vector or ts object.                                                                            |
| <code>order</code>         | Integer vector of length 3: $c(p, d, q)$ . NULL (default) triggers automatic selection.                   |
| <code>seasonal</code>      | A list <code>list(order = c(P,D,Q), period = m)</code> or NULL for automatic selection.                   |
| <code>frequency</code>     | Integer. Required when <code>x</code> is a plain vector.                                                  |
| <code>stepwise</code>      | Logical. Use stepwise search in <code>auto.arima</code> (default TRUE).                                   |
| <code>approximation</code> | Logical. Use approximation in <code>auto.arima</code> (default TRUE).                                     |
| <code>...</code>           | Additional arguments forwarded to <code>forecast::auto.arima()</code> or <code>forecast::Arima()</code> . |

## Value

A named list:

**model\_info** One-row tibble: ARIMA order string, AIC, AICc, BIC, log-likelihood,  $\sigma^2$ .

**coefficients** Tidy tibble with estimate and std error.

**fitted** Tibble: index, observed, fitted, residual.

**diagnostics** Ljung-Box test result tibble.

**model** Raw Arima object.

**Examples**

```
data(AirPassengers)
res = ts_sarima(log(AirPassengers))
res$model_info
```

ts\_sarima\_garch

*Two-Stage SARIMA-GARCH Joint Model***Description**

Stage 1: fits a SARIMA model on the level series via `ts_sarima()`. Stage 2: fits a GARCH model on the SARIMA residuals via `ts_garch()`. Returns both sub-models plus a combined fitted tibble.

**Usage**

```
ts_sarima_garch(
  x,
  sarima_order = NULL,
  sarima_seasonal = NULL,
  garch_order = c(1, 1),
  garch_dist = "std",
  frequency = NULL,
  ...
)
```

**Arguments**

|                              |                                                                    |
|------------------------------|--------------------------------------------------------------------|
| <code>x</code>               | A numeric vector or ts object.                                     |
| <code>sarima_order</code>    | Integer vector <code>c(p,d,q)</code> or NULL (auto).               |
| <code>sarima_seasonal</code> | List <code>list(order=c(P,D,Q), period=m)</code> or NULL.          |
| <code>garch_order</code>     | Integer vector <code>c(p,q)</code> (default <code>c(1,1)</code> ). |
| <code>garch_dist</code>      | Character. Innovation distribution for GARCH (default "std").      |
| <code>frequency</code>       | Integer. Required for plain numeric <code>x</code> .               |
| <code>...</code>             | Additional arguments forwarded to <code>ts_sarima()</code> .       |

**Value**

A named list:

**mean\_model** Result list from `ts_sarima()`.

**variance\_model** Result list from `ts_garch()`.

**fitted** Combined tibble: index, observed, mean\_fitted, sigma, variance, std\_residual.

**model\_info** Combined one-row summary tibble.

**Examples**

```
data(AirPassengers)
res = ts_sarima_garch(log(AirPassengers))
res$model_info
```

---

|        |                                                     |
|--------|-----------------------------------------------------|
| ts_stl | <i>STL (Seasonal + Trend + Loess) Decomposition</i> |
|--------|-----------------------------------------------------|

---

### Description

Wraps `stats::stl()` and returns both a tidy long-format tibble and the raw `stl` object for further use.

### Usage

```
ts_stl(x, frequency = NULL, s_window = "periodic", robust = TRUE, ...)
```

### Arguments

|                        |                                                                                                                    |
|------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>x</code>         | A <code>ts</code> object with <code>frequency &gt; 1</code> , or a numeric vector with frequency specified.        |
| <code>frequency</code> | Integer. Required if <code>x</code> is a plain numeric vector.                                                     |
| <code>s_window</code>  | Seasonal smoothing window passed to <code>stl()</code> . Use "periodic" (default) for purely periodic seasonality. |
| <code>robust</code>    | Logical. Use robust fitting in STL (default TRUE).                                                                 |
| <code>...</code>       | Additional arguments forwarded to <code>stats::stl()</code> .                                                      |

### Value

A named list:

**components** Tidy tibble with columns `index`, `observed`, `trend`, `seasonal`, `remainder`.

**strength** Tibble: `seasonal_strength` and `trend_strength` (Wang et al. 2006 metrics).

**model** Raw `stl` object.

### Examples

```
data(AirPassengers)
res = ts_stl(AirPassengers)
res$components
```

---

|         |                                             |
|---------|---------------------------------------------|
| ts_test | <i>Stationarity Tests for a Time Series</i> |
|---------|---------------------------------------------|

---

### Description

Runs ADF, KPSS, and PP tests and returns a tidy summary data frame.

**Usage**

```
ts_test(
  x,
  adf_lags = NULL,
  kpss_type = c("Level", "Trend"),
  pp_type = c("Z(t_alpha)", "Z(alpha)"),
  verbose = TRUE
)
```

**Arguments**

|                  |                                                                                               |
|------------------|-----------------------------------------------------------------------------------------------|
| <b>x</b>         | A numeric vector, ts object, or single-column data frame.                                     |
| <b>adf_lags</b>  | Integer. Max lag for ADF (default: NULL = auto via <code>trunc((length(x)-1)^(1/3))</code> ). |
| <b>kpss_type</b> | Character. "Level" (default) or "Trend" for KPSS null hypothesis.                             |
| <b>pp_type</b>   | Character. "Z(t_alpha)" (default) or "Z(alpha)".                                              |
| <b>verbose</b>   | Logical. Print test results to console (default TRUE).                                        |

**Value**

A tibble with columns:

|                        |                                                |
|------------------------|------------------------------------------------|
| <b>test</b>            | Test name (ADF / KPSS / PP)                    |
| <b>null_hypothesis</b> | Plain-English description of H0                |
| <b>statistic</b>       | Test statistic                                 |
| <b>p_value</b>         | p-value (or NA when only bounds are available) |
| <b>conclusion</b>      | Character: "Stationary" or "Non-stationary"    |

**Examples**

```
data(AirPassengers)
ts_test(AirPassengers)
```

---

ts\_transform

*Transform a Time Series for Stationarity*


---

**Description**

Applies variance-stabilising transformation (Box-Cox / log) and/or differencing, returning the transformed series together with all parameters needed to back-transform forecasts.

**Usage**

```
ts_transform(
  x,
  method = c("none", "log", "boxcox"),
  lambda = NULL,
  diff = 0L,
  seasonal_diff = 0L,
  frequency = NULL,
  verbose = TRUE
)
```

**Arguments**

|                            |                                                                                                                                          |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>x</code>             | A numeric vector or ts object.                                                                                                           |
| <code>method</code>        | Character. Variance transformation: "none" (default), "log", "boxcox" (auto-selects lambda via <code>forecast::BoxCox.lambda()</code> ). |
| <code>lambda</code>        | Numeric. Fixed Box-Cox lambda. Ignored unless <code>method = "boxcox"</code> . If NULL (default), lambda is estimated.                   |
| <code>diff</code>          | Integer. Number of regular differences (default 0).                                                                                      |
| <code>seasonal_diff</code> | Integer. Number of seasonal differences (default 0).                                                                                     |
| <code>frequency</code>     | Integer. Series frequency, required when <code>x</code> is a plain numeric vector and <code>seasonal_diff &gt; 0</code> .                |
| <code>verbose</code>       | Logical. Print transformation summary (default TRUE).                                                                                    |

**Value**

A named list:

**transformed** Transformed ts object ready for modelling.

**params** Named list with `method`, `lambda`, `diff`, `seasonal_diff`, `frequency` — everything needed by `ts_back_transform()`.

**summary** One-row tibble describing each step applied.

**See Also**

[ts\\_back\\_transform](#)

**Examples**

```
data(AirPassengers)
res = ts_transform(AirPassengers, method = "log", diff = 1,
                  seasonal_diff = 1)
res$summary
plot_ts(res$transformed, title = "Transformed AirPassengers")
```

---

water\_quality

*Water Quality Dataset*

---

**Description**

A dataset containing water quality evaluation metrics for 20 rivers, including dissolved oxygen (O2, positive indicator), pH value (PH, centered indicator), total bacteria count (germ, negative indicator), and plant nutrient content (nutrient, interval indicator with optimal range 10-20). This dataset is suitable for multi-criteria decision analysis, such as weight calculation and fuzzy comprehensive evaluation in the `mathmodels` package.

**Usage**

```
water_quality
```



**Format**

A data frame with 20 rows and 5 columns:

**ID** Numeric, unique identifier for each river (1 to 20).

**O2** Numeric, dissolved oxygen content (mg/L), higher values are better (positive indicator).

**PH** Numeric, pH value, values closer to 7 are optimal (centered indicator).

**germ** Numeric, total bacteria count, lower values are better (negative indicator).

**nutrient** Numeric, plant nutrient content (mg/L), optimal range is 10-20 (interval indicator).

**Details**

Water Quality Dataset

**Source**

Simulated data for water quality evaluation, created for demonstration purposes.

**Examples**

```
# Load the dataset  
data(water_quality)
```

```
# Preview the data  
head(water_quality)
```

# Index

- \* **datasets**
  - water\_quality, 56
- AHP, 3
- basic\_DEA (DEA), 8
- basic\_SBM (DEA), 8
- combine\_preds, 4
- combine\_weights, 4
- compute\_mf, 5
- compute\_mf\_funs (compute\_mf), 5
- coupling\_degree (system\_evaluation), 46
- critic\_weight, 6
- cv\_weight, 7
- DEA, 8
- defuzzify, 10
- DGM21 (grey\_models), 16
- dsigmoid\_mf (membership), 21
- EG (regional\_economics), 44
- entropy\_weight, 11
- epi\_metrics, 12
- fuzzy\_eval, 13
- gauss2mf (membership), 21
- gauss\_mf (membership), 21
- gbell\_mf (membership), 21
- ggplot, 33–35
- gini (inequality), 17
- gini0 (inequality), 17
- GM11 (grey\_models), 16
- GM11\_markov, 14
- GM1N (grey\_models), 16
- grey\_analysis, 15
- grey\_corr (grey\_analysis), 15
- grey\_corr\_topsis (grey\_analysis), 15
- grey\_models, 16
- HHI (regional\_economics), 44
- inequality, 17
- linear\_sum, 19
- LQ (regional\_economics), 44
- malmquist (DEA), 8
- markov\_chain, 20
- membership, 21
- model\_logistic, 24
- model\_lv, 25
- model\_malthus, 26
- model\_seir, 27
- model\_si, 28
- model\_sir, 29
- model\_sis, 30
- normalize (preprocess), 40
- obstacle\_degree (system\_evaluation), 46
- ode\_solver, 31
- pca\_weight, 32
- pi\_mf (membership), 21
- plot\_compartments, 33
- plot\_incidence, 33
- plot\_mf (membership), 21
- plot\_phase\_si, 34
- plot\_Rt\_estimate, 35
- plot\_ts, 35
- plot\_ts\_acf, 36
- plot\_ts\_forecast, 37
- plot\_ts\_garch, 38
- plot\_ts\_pacf, 38
- plot\_ts\_residuals, 39
- plot\_ts\_sarima\_garch, 39
- plot\_ts\_stl, 40
- preprocess, 40
- psigmoid\_mf (membership), 21
- rank\_sum\_ratio, 43
- read\_nbs, 44
- regional\_economics, 44
- rescale (preprocess), 40
- rescale\_extreme (preprocess), 40
- rescale\_initial (preprocess), 40
- rescale\_interval (preprocess), 40
- rescale\_mean (preprocess), 40
- rescale\_middle (preprocess), 40

s\_mf (membership), 21  
sigmoid\_mf (membership), 21  
standardize (preprocess), 40  
super\_DEA (DEA), 8  
super\_SBM (DEA), 8  
system\_evaluation, 46  
  
theil (inequality), 17  
theil0 (inequality), 17  
theil0\_g (inequality), 17  
theil\_g (inequality), 17  
theil\_g2\_cross (inequality), 17  
theil\_g2\_nest (inequality), 17  
to\_positive (preprocess), 40  
topsis, 47  
trap\_mf (membership), 21  
tri\_mf (membership), 21  
ts\_back\_transform, 49, 56  
ts\_ets, 49  
ts\_forecast, 50  
ts\_garch, 51  
ts\_sarima, 52  
ts\_sarima\_garch, 53  
ts\_stl, 54  
ts\_test, 54  
ts\_transform, 55  
  
verhulst (grey\_models), 16  
  
water\_quality, 56  
  
z\_mf (membership), 21